

Software-as-a-Graph: Heterogeneous Graph Learning for Pre-Deployment Dependability Analysis of Asynchronous and Event-Driven Distributed Systems

Ibrahim Onuralp Yigit^{a,*}, Feza Buzluca^a

^a*Department of Computer Engineering, Istanbul Technical University, Istanbul, 34469, Turkey*

Abstract

Modern asynchronous publish–subscribe systems separate components in space and time, but they also obscure how failures spread across components. A key challenge for reliable distributed systems is whether graph representation learning can predict failure impact directly from Architecture-as-Code manifests, without runtime data. In this study, we introduce Software-as-a-Graph (SaG), a static analysis tool that turns deployment manifests into typed multigraphs to help decide when and where to use relational graph learning. We compare a Heterogeneous Graph Transformer with Quality-of-Service edge encodings (HGT-QoS) to traditional network baselines, and include an interpretable ISO/IEC 25010/25019 attribution layer for diagnostic refactoring.

We tested our approach in twelve different scenarios and five real open-source systems. The results show that relational typing and continuous QoS encodings work as substitutes, not as complements. HGT-QoS outperformed untyped networks in new situations ($\Delta\rho = +0.234$, reaching $\rho = 0.638$). In zero-shot transfer tests, HGT-QoS accurately tracked how failures spread in asynchronous event meshes, but performed worse in synchronous RPC call trees where timeouts move up the stack. Since the learned ranking is similar to a simple QoS-weighted centrality baseline, we propose a Dual-Engine Consensus Protocol. Using both models together gives reliable critical-set triage, while the ISO/IEC attribution layer helps find structural root causes for targeted fixes. Although extracting features can take up to 79.3 seconds, which is 2 to 18 times longer than in-process simulation, neural inference is much faster at 56 milliseconds. This shows that static gating can avoid the need for live staging clusters and allows for sub-second gating when combined with incremental topology caching in continuous integration pipelines.

Keywords: Heterogeneous graph neural networks, distributed systems dependability, publish–subscribe architecture, cascading failures, static system analysis, explainable AI

1. Introduction

1.1. Motivation

Modern large-scale distributed software systems progressively use asynchronous, event-driven, publish–subscribe (pub-sub) architectures. These are common within autonomous driving (ROS 2 [1]), enterprise event streams (Apache Kafka [2]), cyber-physical systems (DDS [3]), IoT deployments (MQTT [4]), cloud-native microservices [5, 6], and distributed AI/LLM clusters. Pub-sub architectures separate producers and consumers in terms of space, time, and synchronization [7]. Components communicate indirectly through message topics and brokers, so they do not need direct static references. Current middleware also supports deployment-time Quality-of-Service (QoS) policies, such as reliability, durability, message priorities, and delivery deadlines, to help manage performance during peak loads and network stress.

*Corresponding author

Email addresses: yigiti@itu.edu.tr (Ibrahim Onuralp Yigit), buzluca@itu.edu.tr (Feza Buzluca)

While this separation allows for flexible scaling, it also makes it harder to see what is happening inside the system. Unlike synchronous architectures like RESTful HTTP or gRPC, which show clear caller and callee paths, pub-sub publishers and subscribers lack direct links. As a result, failures, head-of-line blocking, and backpressure can spread along concealed paths involving brokers, shared topics, colocated hosts, and shared libraries [8, 9]. These failures mainly happen in two ways: sequential cascades, where a slow subscriber fills up a broker queue and gradually slows down its publishers [10]; and simultaneous blast radii, where a shared library crash or host outage brings down all colocated services. Standard architecture diagrams and static call graphs cannot show these processes. The best time to reduce these risks is before deployment, especially during design and continuous integration, following dependable computing principles [11, 12]. At these points, runtime telemetry, distributed tracing, and logs are not available. Therefore, architects and Site Reliability Engineers need to identify the most critical components, topics, and links, understand why they matter, and take targeted actions like broker replication, splitting overloaded topics, or isolating shared libraries to reduce risk. Here, systemic criticality means an entity is either a single point of failure that can disrupt downstream connections if it fails, or an error-propagation hub that can cause large cascading outages in the asynchronous message mesh.

By analyzing architecture directly from configuration manifests, pre-deployment Static System Analysis removes the need for staging clusters, active containers, and live fault-injection tools, which reduces infrastructure overhead. However, static analysis is not faster than in-process simulation in terms of CPU time: extracting topology features takes 79.3s for a 520-component mesh and 239.3s for 2,000 components. In contrast, the in-process discrete-event simulator finishes in 0.08–4.5s (Sections 7.5.1 and 8.2). The learned GNN forward pass is quick (56 ms), but static analysis is 2–18 times slower than simulation across twelve scenarios (median 5.6 times, with the largest mesh being the slowest). This shows that static analysis mainly helps by avoiding infrastructure setup, not by being faster, unless it uses incremental topology caching. So why use static graph learning instead of simulating manifests? As explained below, manifest-level simulation needs execution details like publication rates, payload sizes, and queue capacities, which are often missing or uncalibrated during early design and integration. Message simulation also cannot directly assess unrunnable physical hosts or network links without complicated failure models. Static graph learning, by contrast, captures structural patterns and generalizes across both runnable and unrunnable infrastructure.

1.2. Problem Statement: The Architecture–Code Gap and the Black-Box AI Challenge

Pre-deployment dependability analysis in distributed architectures has two main tasks. The first is predictive forecasting: whether a data-driven, relation-specific graph neural network can predict cascading failure blast radii and identify critical components better than traditional topological metrics. While closed-form metrics show global network connectivity, it remains unclear whether they can track multi-hop, relation-dependent cascade spread across several channels. We test this directly against such a baseline (Section 7.1). We train and evaluate the predictive model using independent simulation ground truth to rank and identify critical sets.

Explainable Criticality Attribution (the Explanation Layer) addresses the limits of ranking alone. A ranked list shows where risk is highest, but not how to fix it. To solve this, the predictor is combined with an interpretable structural quality profile based on ISO/IEC 25010 [13] and ISO/IEC 25019 [14]. This layer identifies structural risk types using standard software quality models. For example, it can distinguish between an unreplicated single point of failure, an error-propagating cascade hub, and a maintainability bottleneck. This helps guide targeted architectural fixes. The layer is used only for attribution, not for ranking.

This separation is built into the architecture, not just the presentation: both pathways use the same graph but do not share parameters, and neither is trained on the other’s output. The term that could connect them is off by default and reported only as an ablation (Section 4.2). This independence lets Software-as-a-Graph (SaG) find components that are structurally central but have low operational impact, supporting a more detailed diagnosis than either pathway could provide alone.

The distance between an architecture as designed and as realized is long-established: Perry and Wolf [15] formulated architectural erosion three decades ago, and the architectural-technical-debt literature has tracked it since [16, 17]. What we term the **Architecture–Code Gap** specializes this concept to asynchronous

middleware, where the challenge is that failure semantics were never expressible in artifacts a build pipeline can inspect: *a distributed system can have pristine, bug-free source code within each service, yet remain fragile to ruinous global outages caused by hidden single points of failure (SPOFs) or mismatched middleware Quality-of-Service (QoS) contracts*. This fragility is acute in pub-sub architectures where producers and consumers interact indirectly without static references. Classical architecture evaluations (e.g., ATAM [18, 12]) rely on manual elicitation; static code analysis [19, 20, 21, 22] inspects individual services in isolation; chaos engineering [23] requires provisioned staging clusters; and homogeneous centrality [24, 25, 26, 27] flattens typed topologies into untyped graphs. Section 2 develops each paradigm in detail.

Although machine learning has made big advances in software engineering, current AI methods for system dependability often work as black boxes. Deep neural models usually give risk scores or hidden representations lacking clear, actionable reasons for their predictions. In mission-critical software engineering, this shortage of transparency is not enough. Developers and Site Reliability Engineers need clear explanations of component vulnerabilities and architectural problems so they can refactor code or adjust infrastructure effectively.

1.3. The Software-as-a-Graph (SaG) Approach

To address both the Architecture–Code Gap and the black-box AI problem, this work describes **Software-as-a-Graph (SaG)**, an AI-based pre-deployment **Static System Analysis (SSA)** framework for asynchronous and event-driven distributed systems. SaG provides the tools and experiments needed to determine whether static representations before deployment can identify systemically critical components and explain their systemic root causes. SaG uses a four-stage pipeline: (1) modeling the architecture as a typed, directed multigraph over Applications, Brokers, Topics, Execution Hosts, and Shared Libraries (Section 3.1); (2) mapping physical connections into a QoS-weighted semantic **DEPENDS_ON** layer that captures sequential cascades and simultaneous failures (Section 3.2); (3) training a Heterogeneous Graph Transformer (HGT) to predict multi-hop cascading blast radii (Section 4); and (4) evaluating an explainable Reliability–Maintainability (RM) profile (Section 5) to show why components are fragile.

Importantly, SaG enforces a strict **input–label independence guarantee**: the learned models and attribution baselines use only the analytical graph G_{analysis} . Meanwhile, ground-truth failure impacts come from independent discrete-event simulators running on the raw structural topology $G_{\text{structural}}$ (Section 4.4). This separation keeps feature construction and evaluation ground-truth structurally separate, guaranteeing the method is valid. Section 3 explains the full architectural flow and pipeline interactions (see Figure 1).

The Core Claim and Boundary Conditions. The main claim of this paper is that pre-deployment Architecture-as-Code manifests contain enough relational and QoS information to predict multi-hop cascading failure blast radii and identify critical components, without running code, collecting runtime telemetry, or setting up staging clusters. In particular, relation-specific heterogeneous graph attention (**HGT–QoS**) provides the inductive bias needed to capture indirect cascade spread across message brokers, topics, and shared libraries, even when the distribution changes. However, this claim depends on two empirical boundary conditions:

1. **The Typing–QoS Substitution Effect:** Relational typing and continuous QoS edge encodings act as empirical substitutes rather than additive complements. While each independently produces substantial gains over an unweighted baseline (+0.134 and +0.187, Holm $p = 0.0015$), combining both exhibits diminishing returns because both channels supply redundant indicators of edge relation identity (Section 7.2).
2. **Communication Paradigm Alignment:** Relational inductive bias transfers zero-shot to asynchronous publish–subscribe event meshes (where failure propagation flows forward via message starvation), but reverses on synchronous RPC call trees (where failures cascade backward via timeouts and retries), establishing that graph directional biases must correspond to architectural communication semantics (Section 7.4).

Operational Guidance and Method Portfolio. To help with effective adoption, SaG offers a set of methods customized to different software engineering situations:

- **HGT-QoS (Heterogeneous Learning):** Recommended for complex asynchronous publish-subscribe event meshes (e.g., ROS 2, DDS, MQTT) where cascading failures propagate through multi-hop broker topologies, shared libraries, and hardware hosts under unseen architecture topologies.
- **Topo-QoS (Closed-Form Centrality):** Recommended for lightweight, zero-training CI/CD gates that require instantaneous deterministic scoring without GPU resources or model training cost ($\rho = 0.553$).
- **Dual-Engine Consensus Protocol:** Recommended for production pre-deployment gating (Section 8.1). Because label-free uncertainty cannot reliably predict when an ML model struggles out-of-distribution ($\rho_s = -0.126$), SaG evaluates HGT-QoS and Topo-QoS concurrently: components in the unanimous top- K intersection receive high-confidence triage, while ranking divergences trigger architectural review.
- **Explanation Layer ($Q(v)$ - ISO/IEC 25010):** Applied post-ranking as a diagnostic attribution instrument rather than a continuous ranker (Section 5), decomposing flagged components into standardized Reliability (Availability vs. Fault Tolerance) and Maintainability profiles to guide targeted refactoring (e.g., broker replication vs. circuit breakers).

Rationale for Graph Learning vs. Direct Simulation. Since discrete-event simulation $I^*(v)$ gives the ground-truth criticality as well as finishes in 0.08–4.5s, it is important to explain why we train a graph model. Manifest analysis avoids the need for staging clusters and chaos testing tools—this is about avoiding extra infrastructure, not saving energy (Section 8.2 covers the energy used by the analysis itself). There are three main technical reasons to use graph learning instead of direct manifest simulation:

1. **Scoring Unsimulated Infrastructure Entities:** While discrete-event simulation sweeps message-passing components (Applications, Brokers, Libraries) across topics, it cannot directly evaluate unrunnable physical Execution Hosts or hardware interconnects without intricate physical failure models. Message passing over heterogeneous multigraphs naturally propagates relational representations across both simulated and unsimulated entities.
2. **Robustness to Operational Parameter Drift:** Direct simulation depends on declared publication rates, payload sizes, and queue limits, which are often uncalibrated or only estimates during early design. Graph learning, however, captures topology-based and QoS patterns that extend across different architectures, even when operational details are not fully set.
3. **Sub-Second Continuous Integration with Incremental Caching:** In CI/CD, using incremental caching limits feature extraction to the k -hop neighborhood around pull-request changes, allowing the 56 ms forward pass for almost instant pre-commit checks. Without caching, static analysis is still slower than direct simulation in all tested scenarios.

Section 7.1 evaluates whether graph learning offers ranking advantages over closed-form baselines.

1.4. Research Questions

This empirical study considers five research questions:

- **RQ1 (Predictive Efficacy):** *How accurately does heterogeneous graph learning predict cascading failure impact and identify the critical component set, compared with traditional, non-learning network indicators?*
- **RQ2 (Value of Architectural Typing):** *Does modeling distinct entity and dependency types yield better failure predictions than homogeneous graph models on architectures the model has never seen — and does whatever advantage it confers depend on what other relational signal the model already has?*
- **RQ3 (QoS Encoding and Robustness):** *(i) Do middleware Quality-of-Service contracts carry signal a purely structural score discards, (ii) does that signal compose with or substitute for architectural typing, (iii) do the framework’s simulation oracles agree with one another, and (iv) are the reported orderings robust to the free parameters of the scorer and of the ground truth?*
- **RQ4 (Real-World Generalization):** *How effectively does the framework transfer zero-shot to authentic, real-world distributed systems across autonomous driving (ROS 2), cloud-native microservices, smart home IoT, and industrial edge computing?*

- **RQ5 (Analysis Cost):** *What does pre-deployment analysis cost at CI/CD time, which pipeline stage dominates that footprint, and how does it compare against the discrete-event simulation it is intended to displace?*

1.5. Key Contributions

This paper makes three main contributions:

1. **Heterogeneous Graph Learning for Dependability, and Its Boundaries:** A relation-specific Heterogeneous Graph Transformer that forecasts cascading blast radii from Architecture-as-Code manifests, incorporating 16-D QoS edge encodings and an auxiliary component reliability head (Section 4). Ablated separately under inductive distribution shift across twelve architectures, relation-specific HGT demonstrates out-of-distribution gains ($\Delta\rho = +0.234$ over an untyped baseline, $p = 0.0005$, reflecting the joint transition to the parameterized bidirectional HGT architecture). Crucially, typing and continuous QoS encodings act as empirical substitutes rather than complements — an interaction that survives transformation to a variance-stabilized scale, so it is a property of the mechanisms rather than of bounded ρ — though the typing factor also carries a $15.4\times$ capacity gap and a change in message-passing directionality that the reported controls do not separate. Against an unparameterized QoS-weighted centrality baseline, learned ranking does not establish a statistically significant advantage ($+0.085$, $p = 0.151$). That parity should be read against the target: $I^*(v)$ is a deterministic functional of the same graph the predictors read, so a closed-form centrality is an expected-strong comparator. The result bounds this design more than it bounds graph learning. We report the substitution effect and the empirical boundary as primary findings (Sections 7.1–7.2).
2. **A Formal Typed Architecture Model:** A multigraph representation that derives logical dependencies from physical pub-sub linkages and distinguishes sequential cascade propagation from simultaneous multi-consumer library failures (Section 3).
3. **Empirical Benchmark, Real-World Transfer, and Cost Profile:** A study across seventeen system architectures totaling 2,812 components: twelve synthetic topologies (2,461 components across LOSO folds) and five open-source reference systems (351 components) under strict graph-view separation. We characterize pipeline costs and show that the neural forward pass accounts for only 0.02% of runtime. In contrast, cold deterministic feature extraction dominates and costs $2\text{--}18\times$ the simulation it was meant to displace (median $5.6\times$ over twelve scenarios, 79.3s against 4.5s at its maximum), refuting the premise that static analysis is inherently faster than in-process simulation without incremental caching (Sections 6–7).

One important point about these results should be clear before quoting them. The learned figures in this paper are sensitive to revisions: between the two most recent runs, using the same seeds and data, the training-free results are identical, but the learned fold means can change by up to 0.172—twice the $+0.085$ margin that the main comparison relies on (Section 8.3). The direction of this difference is stable in all our tests, but its size depends on the specific artifacts we provide, not just the method.

Relationship to the authors’ prior work. A previous conference paper [28] introduced the preliminary multigraph formulation and deterministic quality model on synthetic topologies. This JSS manuscript substantially extends that work through incorporating the complete predictive HGT pathway with 16-dimensional QoS edge encoding (Section 4); inductive LOSO cross-validation (Section 7.2); zero-shot evaluation throughout five open-source systems (Section 7.4); empirical cost and sustainability characterization (Section 7.5); multi-oracle convergent validity and graph-view separation (Sections 4.3–4.4); and system-wide sensitivity analyses (Section 7.3). We retained formalisms from the conference paper only in restructured portions of Sections 3 and 5. Two of its conclusions do not survive the re-measurement reported here. We state that explicitly rather than leaving the earlier record standing: the deterministic quality model is not a competitive ranking instrument on this corpus ($\rho = 0.205$, below unweighted centrality, Section 7.1), and its elicited weights rank worse than a uniform prior (Section 7.3). The corpus, the oracles, and the evaluation population all differ from [28], so these are corrections to what we now believe rather than a failure to reproduce a fixed experiment.

1.6. Paper Organization

The rest of this paper is organized as follows: Section 2 reviews related work. Section 3 explains the SaG multigraph model and dependency projections. Section 4 describes the Heterogeneous Graph Transformer and simulation oracles. Section 5 presents the ISO/IEC-based explanation layer. Section 6 covers the experimental protocol, and Section 7 reports results for RQ1–RQ5. Section 8 discusses practical consequences, sustainability, threats to validity, and limitations. Section 9 concludes.

2. Related Work

This study brings together four main research areas: dependability, performance, and sustainability in distributed software systems; static code and system analysis; software quality analysis and multi-criteria evaluation; and graph representation learning with explainable AI. Together, these fields provide the basis for predicting cascading vulnerabilities from Architecture-as-Code descriptors before setting up any infrastructure.

2.1. Dependability, Performance, and Sustainability in Distributed Software Systems

Publish-subscribe (pub-sub) and asynchronous event-driven models separate communicating entities by space, time, and synchronization, enabling scalable, high-throughput systems [7]. Modern middleware standards like ROS 2 [1], Apache Kafka [2], DDS [3], and MQTT [4] manage these exchanges using detailed Quality-of-Service (QoS) policies. In cloud-native microservice meshes and distributed AI or LLM serving systems, asynchronous message passing and queueing are the main ways components communicate, affecting latency, throughput, and hardware use.

Previous research on dependability and performance has focused on runtime methods such as dynamic consensus, broker clustering, backpressure throttling, autoscaling, and automated failover. It has also explored chaos engineering and runtime fault injection [23], which intentionally disrupt live clusters to study degradation and recovery. While runtime testing is realistic, it requires a working cluster, can disrupt services, and uses significant computing resources. These factors make it less practical during early design or commit-level CI/CD, and it is one of the development-time computations examined in green software engineering [29, 30, 31, 32, 33]. By contrast, this study examines dependability using Architecture-as-Code descriptors before setting up any infrastructure.

A long history of research has predicted dependability from architectural descriptions. For example, Cheung’s absorbing-Markov-chain model [34] calculates system reliability using component reliabilities and control flow graphs. Goseva-Popstojanova and Trivedi [35] organized later state-based, path-based, and additive models, as reviewed by Immonen and Niemelä [36]. Model-driven frameworks like the Palladio Component Model [37] and layered queueing networks [38] use component specifications to predict performance and reliability. Annotation-based methods, such as the AADL Error Model Annex [39], create fault trees from declared error states. However, these approaches cover broader topics than this study and need operational profiles and failure rates that are not available at commit time. Instead, SaG focuses on a specific question using only manifests: which components’ failures spread the furthest in the declared topology?

Many studies find faults in microservices by using operational telemetry. For example, Seer [40] and Sage [41] predict QoS problems from hardware counters and traces. MicroRCA [42] and TraceRCA [43] find root causes using service-dependency graphs. DeepTraLog [44], Eadro [45], and MicroCause [46] use graph neural networks on traces, logs, and metrics, as Zhang et al. [47] review. Zhou et al. [48] found that cascading outages in synchronous microservices often stem from thread-pool exhaustion, RPC timeouts, and repeated retry storms along call trees. In pub-sub systems, failures spread through broker queue saturation and message starvation. Because these methods require a running cluster to produce runtime telemetry, SaG works before deployment by analyzing static manifests before any code runs.

2.2. Static Code Analysis (SCA) vs. Static System Analysis (SSA)

Traditional **Static Code Analysis (SCA)** tools (e.g., SonarQube [19]) inspect source code Abstract Syntax Trees (ASTs) within individual services. They evaluate cyclomatic complexity [20], class cohesion, module coupling (e.g., Lack of Cohesion in Methods [LCOM], Coupling Between Objects [CBO]) [21, 22], and

code duplication to flag internal code smells and defect-prone modules [49, 50, 51, 52]. However, SCA cannot observe runtime communication topology: it does not capture inter-service messaging channels, message broker queue saturation, or cross-host failure propagation.

Recovering system-level structure through static analysis is an active research area. Bushong et al. [53] create communication diagrams from static code analysis, and a recent review looks at nine architecture recovery tools [54]. In robotics and publish-subscribe systems, tools like HAROS [55] use static analysis to find structural problems before launching the system. Together, this research helps build accurate system models for better understanding and drift detection. Unlike these, SaG uses the declared topology to predict how failures might spread.

To close the "Architecture–Code Gap," Static System Analysis (SSA) expands static analysis from individual service code to the whole system architecture. By modeling distributed applications, message topics, brokers, execution nodes, and shared libraries as a connected multigraph, SSA spreads code-level quality metrics across architectural dependencies. As a result, engineering teams can find structural anti-patterns [56, 57] and architectural technical debt [58] early in the CI/CD process [59, 60], before flawed topologies reach production.

2.3. Software Quality Models and Multi-Criteria Evaluation

The ISO/IEC 25010:2023 product quality model [13] and the ISO/IEC 25019:2023 Quality-in-Use model [14] define software product quality. From the characteristics listed under Reliability, Maintainability, and Efficiency of Performance in ISO/IEC 25010:2023, SaG focuses on those derived from deployment topology: Availability, Fault Tolerance, Modularity, Modifiability, and Analyzability (Section 5.1). Accordingly, topological analysis does not cover other qualities such as Faultlessness, Recoverability, Reusability, and Testability.

Software engineering measures both internal quality, which refers to built-in structural features checked on static artifacts, and external quality, which covers runtime dependability and behavior during system operation [61, 62]. In distributed systems, technical debt such as over-centralized message topics or unreplicated brokers can lower internal quality and lead to serious external problems, including performance bottlenecks, queue congestion, and outages.

Combining different structural metrics into a single, auditable quality score is a classic Multi-Criteria Decision Making (MCDM) problem. The Analytic Hierarchy Process (AHP) [63] uses structured pairwise comparisons and a Consistency Ratio to ensure consistent judgments ($CR \leq 0.10$). However, while this ratio finds inconsistencies, it cannot spot matrices filled with preset answers, which this study addresses for the weights (Section 5.2). Therefore, AHP provides an audited, explainable Reliability–Maintainability (RM) quality baseline alongside learned graph models.

2.4. Graph Representation Learning and Explainable AI

Network science uses centrality indices such as degree, closeness, betweenness centrality [24, 26], articulation points, and PageRank [25, 27] to identify important nodes. Key studies on network robustness [10], cascading overloads [8], and interdependent networks [9] model how disruptions spread in connected systems. Percolation models are useful for comparing network breakdowns, but their thresholds assume homogeneous, undirected networks. Because these calculations do not apply to directed multigraphs with different entity types, channels, and QoS contracts, this study uses centrality-based baselines (Section 6.2). It suggests multi-type percolation fragmentation as a promising future approach.

Standard network measures have two main problems when applied to software architectures. First, Dimensional Collapse happens when a single centrality score cannot show why a component is critical, such as telling apart a single point of failure, a cascade hub, or an overused library. Second, Semantic Collapse occurs when unweighted metrics treat all nodes and edges the same, mixing up very different architectural elements like message topics, shared libraries, and physical hosts. Together, these problems motivate the core claim that richer architectural analysis is needed.

To overcome the limits of hand-crafted metrics, recent studies use machine learning for network vulnerability analysis and ranking node importance. Examples include FINDER [64], which applies deep reinforcement

learning to find key nodes that break network connectivity; DrBC [65], which uses Graph Neural Networks to estimate betweenness centrality in large networks; and PowerGraph [66], which models power grid risks. However, these models assume homogeneous, undirected, unweighted graphs. They cannot be used directly for distributed software systems because Architecture-as-Code manifests are heterogeneous: software topologies are directed multigraphs with several entity types, multiple relation types, and complex QoS contracts. Flattening these architectures into single-layer graphs for DrBC or FINDER causes severe Semantic Collapse and removes important edge contracts. As a result, existing homogeneous models (GCN [67], GraphSAGE [68], GAT [69]) average signals across all connection types and generalize poorly. Heterogeneous Graph Neural Networks (RGCN [70], HAN [71], HGT [72], MAGNN [73]) solve this by using relation-specific transformations. This study uses the Heterogeneous Graph Transformer (HGT) [72] with edge encodings to keep relational semantics when predicting cascade blast radii, and also tests matched homogeneous models (GAT-N, GAT-N-QoS) and closed-form baselines (Topo-QoS) to compare learning approaches (Section 6.2). Graph learning has also been used for microservice topologies; for example, Khodabandeh et al. [74] predict future service interactions using graph attention on segmented call graphs. However, that method predicts edge existence based on past interactions. In contrast, this study uses a declared topology as input and predicts the blast radius after removing a node.

A key challenge in using modern AI for software engineering is the black-box problem: deep neural models produce risk scores or embeddings without explaining the underlying causes. In real-world settings, these unclear risk rankings make it hard for developers and site reliability engineers (SREs) to decide whether to replicate hosts, set up circuit breakers, or refactor shared libraries.

Current graph neural network (GNN) explanation methods, such as GNNExplainer [75] and PGExplainer [76], identify important subgraphs by masking edges or using parameterized learning. While useful, these methods explain models using internal features instead of standard software engineering terms. SaG addresses this by using a dual-pathway design. The predictive HGT pathway shows which architectural relations spread the cascade by revealing mutual-attention patterns (Section 7.3.3). Meanwhile, the explanation layer links fragility to ISO/IEC quality sub-characteristics (Section 5), turning predictions into practical and cost-effective solutions.

3. The Software-as-a-Graph (SaG) Architectural Model

Figure 1 presents the end-to-end architecture of the SaG framework. The shared front end processes Architecture-as-Code manifests, constructs a typed multigraph, projects implicit runtime interactions throughout a QoS-weighted logical dependency layer, and extracts typed node properties. These features feed two independent pathways with no shared parameters. The predictive pathway (Section 4) forecasts cascading failure blast radii using a Heterogeneous Graph Transformer. The explanation layer (Section 5) decomposes fragility into Reliability and Maintainability quality profiles.

This section formalizes the Software-as-a-Graph multigraph representation (Section 3.1), the QoS-aware weighting and logical dependency derivation rules (Section 3.2), the dual graph views (Section 3.3), and the typed node feature encodings (Section 3.4).

3.1. Formal Multigraph Definition

A complex distributed software system can be explicitly described as a typed, weighted, and directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

where:

- V is the set of system entities, divided into five categories: $\tau_V = \{\text{app}, \text{broker}, \text{topic}, \text{host}, \text{lib}\}$, so $V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{host}} \cup V_{\text{lib}}$. To avoid confusion with physical machines, V_{host} refers to *Execution Hosts*, which can be physical or virtual nodes.
- E is the set of directed edges connecting entities.

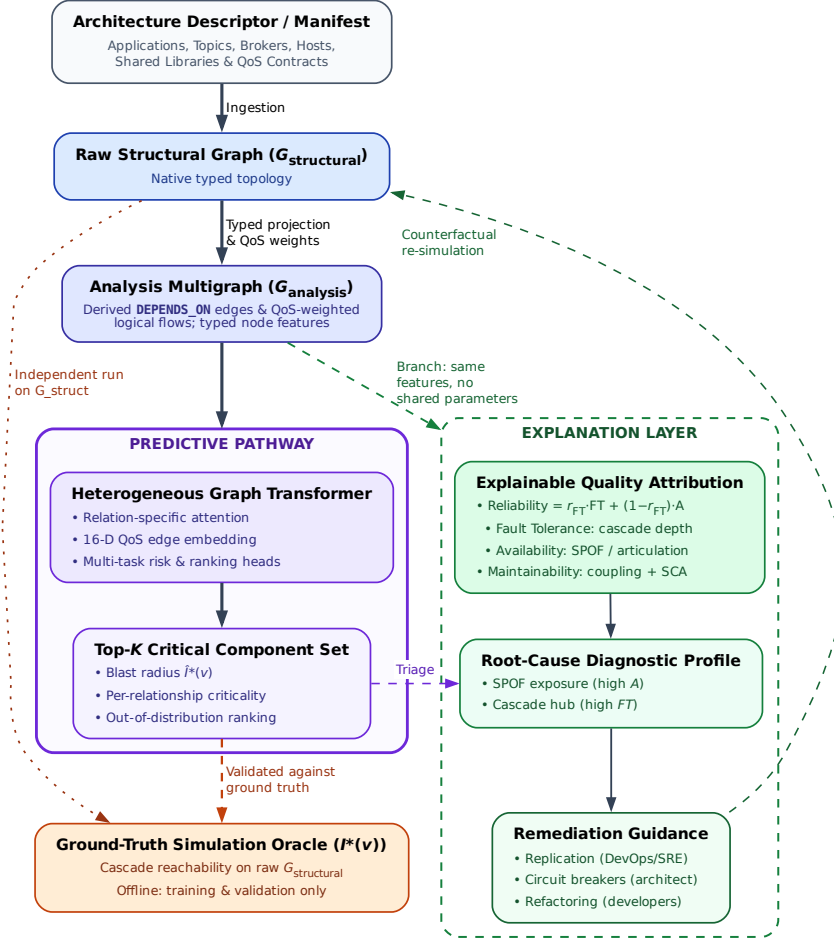


Figure 1: End-to-end architecture of the SaG framework. The predictive pathway runs left to right: manifest ingestion, typed multigraph, QoS-weighted DEPENDS_ON projection, typed node properties, heterogeneous graph learning, ranked critical set. The dashed edge marks the ground-truth simulation oracle, which operates only on $G_{\text{structural}}$, trains the predictor offline and takes no part in inference. The explanation layer re-enters from the analysis multigraph and shares no parameters with the predictor, reaching flagged components through triage rather than data flow.

- $\tau_V : V \rightarrow \mathcal{T}_V$ and $\tau_E : E \rightarrow \mathcal{T}_E$ are typing functions assigning entity and relationship categories.
- $w_V : V \rightarrow (0, 1]$ and $w_E : E \rightarrow (0, 1]$ are weighting functions that show how critical an entity is and how strong a connection is. For applications and shared libraries, $w_V(v)$ is set using static code metrics: $w_V(v) = 1 - \text{CQP}(v)$ (Section 3.4). When static code metrics are unavailable, or for infrastructure entities, set the weight to 1.0 ($w_V(\text{host}) = 1.0$).

Table 1 summarizes the five entity types and six structural edge types in the SaG model, together with their semantics and representative distributed-system implementations.

Application and Library entities use static code metrics from Static Code Analysis (SCA) tools, including lines of code, cyclomatic complexity, coupling between objects, and method cohesion. Together, these metrics show how code-level fragility connects to topological analysis.

3.2. QoS-Aware Weights and Logical Dependency Derivation

In distributed middleware, a communication link’s strength depends on Quality-of-Service (QoS) contracts. For example, a RELIABLE topic with TRANSIENT_LOCAL durability creates a stronger connection between services than a BEST_EFFORT telemetry stream. This difference underpins the following weighting rules.

Table 1: Entity and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	Architectural Role	Concrete System Examples
Application (V_{app})	Autonomous process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	<code>/sensor/lidar</code> , <code>orders.payment.completed</code>
Execution Host (V_{host})	Physical host or virtualized execution environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or runtime dependency	Kafka client, OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	Direction	Semantic Meaning
PUBLISHES_TO	App/Library $\rightarrow$ Topic	Component publishes messages to topic
SUBSCRIBES_TO	App/Library $\rightarrow$ Topic	Component consumes messages from topic
ROUTES	Broker $\rightarrow$ Topic	Broker manages and routes topic traffic
RUNS_ON	App/Broker $\rightarrow$ Host	Process is hosted on physical/virtual host
CONNECTS_TO	Host $\rightarrow$ Host	Physical network link between hosts
USES	App $\rightarrow$ Library	Application links to shared library dependency

Notation. Entity and edge types: Table 1. Simulation oracles: Table 3.

$G_{\text{structural}}$	Raw multigraph; oracles only	$Q(v)$	RM composite quality score
G_{analysis}	DEPENDS_ON projection; predictor input	ρ	Spearman ρ , full population
V_{app}	Application nodes; the scored population	$\rho_{>0}$	Spearman ρ , active stratum ($I^* > 0$)
$w(t), w(e)$	QoS topic weight, edge weight	$F_1@K$	Critical-set overlap, $K = 0.20 V_{\text{app}} $
$I^*(v)$	Primary cascade-reachability oracle	$I_{\text{comp}}, I_{\text{dyn}}, I_M$	Further oracles (Table 3)

Each topic t carries an intrinsic criticality weight $w(t) \in (0, 1]$ combining its declared QoS semantics with two runtime-stress modulators: payload size and publication frequency:

$$w(t) = \alpha_{\text{top}} \cdot \text{QoS}(t) + \beta_{\text{top}} \cdot \text{SizeNorm}(t) + \gamma_{\text{top}} \cdot \text{FreqNorm}(t), \quad (\alpha_{\text{top}}, \beta_{\text{top}}, \gamma_{\text{top}}) = (0.75, 0.15, 0.10) \quad (2)$$

where $(\alpha_{\text{top}}, \beta_{\text{top}}, \gamma_{\text{top}})$ is a convex combination satisfying $\alpha_{\text{top}} + \beta_{\text{top}} + \gamma_{\text{top}} = 1.0$. The QoS term is an AHP-weighted aggregate of the declared contract:

$$\text{QoS}(t) = w_{\text{rel}} \cdot q_{\text{rel}} + w_{\text{dur}} \cdot q_{\text{dur}} + w_{\text{prio}} \cdot q_{\text{prio}}, \quad (w_{\text{rel}}, w_{\text{dur}}, w_{\text{prio}}) = (0.24, 0.62, 0.14) \quad (3)$$

Here, $q_{\text{rel}}, q_{\text{dur}}, q_{\text{prio}} \in [0, 1]$ represent normalized reliability, durability, and transport-priority scores mapped from manifest policies: $q_{\text{rel}} \in \{0.0, 1.0\}$ (best-effort vs. reliable), $q_{\text{dur}} \in \{0.0, 0.5, 1.0\}$ (volatile, transient-local, persistent), and $q_{\text{prio}} \in \{0.0, 0.5, 1.0\}$ (low, medium, high). Durability carries the highest weight because it determines state maintenance and historical message replay across node restarts. We distinguish this structural coupling weight $w(t)$ from the instantaneous cascade reachability oracle $I^*(v)$ (Section 4.3): while $I^*(v)$ scales immediate reachability loss via reliability and priority tiers, durability governs whether failure states persist and propagate across lifecycle boundaries. The sub-weights derive from a Saaty pairwise matrix with consistency ratio $CR = 0.016 \leq 0.10$. We note that CR is uninformative for a matrix back-filled from a chosen priority vector, so we state which case this is: the Topic QoS matrix is one of the two in the framework that carry genuine second-eigenvalue spread. Its CR therefore reports consistency rather than construction (Supplementary Table S4). The modulators $\text{SizeNorm}(t)$ and $\text{FreqNorm}(t)$ are logarithmically compressed and clamped to $[0, 1]$.

$$\text{SizeNorm}(t) = \min \left(1.0, \frac{\log_2(1 + B(t))}{20} \right), \quad \text{FreqNorm}(t) = \min \left(1.0, \frac{\log_{10}(1 + F(t))}{3} \right) \quad (4)$$

Here, $B(t)$ is the message payload size in bytes. The design envelope is 1 MiB, which is the practical DDS sample limit before RTPS fragmentation becomes a problem. $F(t)$ is the normal publication frequency in Hertz. The final weight $w(t)$ is limited to $[0.01, 1]$ so that best-effort edges remain visible during graph traversals. Each structural communication edge connected to t (PUBLISHES_TO, SUBSCRIBES_TO, ROUTES) uses $w_E(e) = w(t)$ and the topic's QoS vector. The full $(\alpha_{\text{top}}, \beta_{\text{top}}, \gamma_{\text{top}})$ simplex keeps the orderings ($\rho \geq 0.919$), with small ranking changes of 0.031 (Topo-QoS) and 0.007 (RM). Morris screening shows that none of the three parameters are critical ($\mu^* \leq 0.025$), so the split is a documented choice, not a sensitive parameter.

3.2.1. Logical Dependency Projection (*DEPENDS_ON*)

Structural edges represent explicit deployment connections but do not capture implicit runtime dependencies. For example, a subscriber depends on a publisher, yet no direct edge connects them in publish-subscribe architectures. To tackle this limitation, a single unified semantic relation, *DEPENDS_ON*, is derived and directed from *dependent* to *dependency* ("if target fails, source is impacted"), following the six projection rules detailed in Table 2. The resulting weight $w \in (0, 1]$ quantifies the magnitude of operational coupling and indicates the conditional likelihood that a disruption in the dependency propagates to the dependent.

Table 2: The six *DEPENDS_ON* logical dependency projection rules.

Rule	Dependency Category	Structural Pattern (Dependent $\rightarrow$ Dependency)	Derived Weight (w)
1	app_to_app	Subscriber $\rightarrow$ Publisher (via shared Topic, incl. transitive USES)	$1 - \prod_{t \in T} (1 - w(t))$
2	app_to_broker	Publisher/Subscriber $\rightarrow$ Broker routing its topics	$1 - \prod_{t \in T} (1 - w(t))$
3	host_to_host	Host $\rightarrow$ Host (lifted from inter-host app dependencies)	$\max_{u \in \text{hosted}(h_1), v \in \text{hosted}(h_2)} w_{\text{DEPENDS_ON}}(u \rightarrow v)$
4	host_to_broker	Host $\rightarrow$ Broker (lifted from hosted app dependencies)	$\max_{u \in \text{hosted}(h)} w_{\text{DEPENDS_ON}}(u \rightarrow b)$
5	app_to_lib	Application $\rightarrow$ Shared Library it USES	$H(w_V(\text{app}), w_V(\text{lib}))$
6	broker_to_broker	Broker $\leftrightarrow$ Broker (shared physical fault-domain colocation, symmetric)	$w_V(\text{host})$

Rules 1 and 2 combine the set of topics T connecting a pair of components using a probabilistic union instead of a maximum [77, 78, 79]. This method confirms that more parallel failure paths always increase coupling, while keeping w within $(0, 1]$. Rule 5 uses the harmonic mean $H(x, y) = 2xy/(x + y)$ [80] to combine the weights of the Application and the shared Library, balancing their importance. Rules 3 and 4 extend dependencies across hosts by using the highest coupling weight.

3.2.2. Sequential Cascades vs. Simultaneous Blasts

A central principle of the SaG model is the distinction between two degradation modes. In a **Sequential Cascade (Rule 1)**, a failed publisher starves downstream subscribers sequentially through message queues and topic buffers. In a **Simultaneous Blast (Rule 5)**, a crashed library or execution host causes all consuming applications and colocated brokers to fail instantaneously in a single shared-fate event. Preserving entity types and relation-specific projection rules lets the SaG model represent both mechanisms, whereas untyped homogeneous graphs collapse them into indistinguishable edges.

Rule 6 is the only symmetric rule: two brokers colocated on the same host share that host's physical failure domain ($w = w_V(\text{host})$). Colocated brokers compete for resources, and host crashes halt them simultaneously. Rule 6 applies in four of the eight scenarios of a companion study's detection benchmark (12 directed edges); that suite is not this paper's corpus, and the figure is quoted only to indicate how rarely the rule fires. Because simulation oracles operate strictly on $G_{\text{structural}}$ (Section 4.4), Rule 6 has zero influence on ground-truth failure labels.

3.3. Dual Graph Views and Architectural Layers

The SaG framework consists of two distinct representations. The **Structural Graph** ($G_{\text{structural}}$) is the raw deployment graph containing physical and structural relations (*PUBLISHES_TO*, *ROUTES*, *RUNS_ON*, *USES*), preserving the untransformed deployment topology. The **Analysis Graph** (G_{analysis}) is the projected graph containing derived *DEPENDS_ON* edges annotated with QoS weights and ingested SCA metrics. We compute all GNN embeddings and analytical metrics on the analysis graph (Supplementary Figure S3).

G_{analysis} is further organized into four analytical layers (Application, Middleware, Infrastructure, and Global System), enabling criticality evaluation at subsystem levels consistent with hierarchical frameworks such as MIL-STD-498 [81].

3.4. Typed Node Feature Encoding

Both the predictive pathway (Section 4) and the explanation layer (Section 5) read the same typed node properties from G_{analysis} : the predictor projects them per entity type before message passing, the explanation layer aggregates them into a quality profile. All five entity types share a common, deterministic 18-dimensional base block (indices 0–17) of topological metrics: (0) PageRank (*PR*), (1) Reverse PageRank (*RPR*), (2) Betweenness Centrality (*BT*), (3) Closeness Centrality (*CL*), (4) Eigenvector Centrality (*EV*), (5) In-Degree Centrality (*DG_{in}*), (6) Out-Degree Centrality (*DG_{out}*), (7) Local Clustering Coefficient (*CC*),

(8) Undirected Articulation Score (AP), (9) Bridge Ratio (BR), (10) Node QoS Weight (w), (11) QoS-Weighted In-Degree (w_{in}), (12) QoS-Weighted Out-Degree (w_{out}), (13) Multi-Path Coupling Index ($MPCT$), (14) Path Complexity (PC), (15) Fan-Out Criticality (FOC), (16) Directed Articulation Point (AP_c^{dir}), and (17) Connectivity Degradation Index (CDI). The full mathematical schema, definitions, and normalizations for each of these eighteen metrics are detailed in Supplementary Table S11 (Supplementary Section S11). CDI is by far the most expensive of these and dominates the analysis cost of Section 7.5; because it is a predictor input and not only a term of the Availability score, it cannot be gated away without changing both pathways. Every metric in the block is normalized to $[0, 1]$ within its graph, which stops raw graph size from driving the per-type projections under cross-scenario transfer. Type-specific blocks extend the vector to 19–25 dimensions, adding source-code metrics and the Code Quality Penalty for Applications, reverse-USES blast-radius drivers for Libraries, queue capacity for Brokers, publisher/subscriber counts and ordinal QoS criticality for Topics, and CPU and memory allocation for Execution Hosts.

The shared block provides the GNN with global structural and positional context—analogous to positional and structural encodings in Graph Transformers—enabling relational message passing to modulate multi-hop representations based on global network role rather than local immediate adjacency alone. Crucially, because betweenness, closeness, reverse PageRank, and articulation scores are topological summaries computed before model evaluation, a learned model is not the only way to derive a criticality score from structure. This design guarantees that the closed-form baselines serve as fair, competitive comparators rather than strawman alternatives, while making deterministic feature extraction the dominant computational bottleneck ($O(|V|^2 + |V||E|)$) analyzed in Section 7.5.

4. Graph Learning for Failure-Impact Prediction

Cascading failure impact in distributed software systems indicates non-linear, multi-hop, and relation-dependent characteristics. Outages propagate through architectural relations and dependencies beyond immediate neighbors. Whether a closed-form combination of standard centrality indices can adequately capture these complicated dynamics remains an open empirical question. Consequently, the primary predictive approach described in Section 1.2 utilizes a learned graph model, which is evaluated in Section 7.1 against a closed-form baseline. The learned model does not manifest significant improvement over the baseline in out-of-distribution ranking.

This section presents the Heterogeneous Graph Transformer (HGT) architecture and its typed edge encodings (Section 4.1), the multi-task prediction heads and dimension-masked loss formulation (Section 4.2), the ground-truth simulation oracles (Section 4.3), and the input–label independence guarantee designed to prevent data leakage (Section 4.4).

4.1. Heterogeneous Graph Transformer Architecture

Distributed systems comprise heterogeneous entity types (Applications, Libraries, Brokers, Topics, Execution Hosts) and diverse interaction semantics (PUBLISHES_TO, SUBSCRIBES_TO, ROUTES, RUNS_ON, CONNECTS_TO, USES, DEPENDS_ON). We therefore employ a three-layer **Heterogeneous Graph Transformer (HGT)** architecture [72], implemented within PyTorch Geometric [82], with hidden dimension $D = 64$ and $H = 4$ attention heads. This architecture guarantees that typed relations, rather than simple adjacency, govern failure-impact forecasting.

4.1.1. Continuous-Categorical Edge Feature Encoding (16- D)

To capture continuous QoS constraints and channel semantics, SaG encodes each directed edge $e = (u, v)$ as a 16-dimensional continuous-categorical vector $e_{uv} \in \mathbb{R}^{16}$. Index 0 is the scalar coupling weight $w_E(e) \in (0, 1]$ of Section 3.2; index 1 is the normalized count of simple paths through e ; indices 2–8 one-hot encode the seven structural and derived relations; and indices 9–15 carry middleware QoS parameters on PUBLISHES_TO and SUBSCRIBES_TO edges, zeroed elsewhere. Six QoS dimensions are active in our corpus — reliability, durability, message priority, a heterogeneity flag raised when an edge’s QoS triple departs from its scenario’s modal profile, and the deadline pair (an active flag and $\log_{10}(1 + \text{deadline_ns}/10^6)$, populated on 463 of 615

topics 75%). The seventh, $\log_{10}(1 + \max_blocking_ms)$, is a schema provision for hard real-time DDS and ROS 2 profiles and is zero throughout.

An edge projection module maps e_{uv} into the hidden space: $e'_{uv} = W_{\text{edge}}e_{uv}$. Before relational attention computation, the current projection vector is incorporated directly into the target node representation: $\tilde{h}_v = h_v + e'_{uv}$.

4.1.2. Type-Specific Projection and Heterogeneous Message Passing

For each source node u and target node v connected by meta-relation $\tau(e) = (\tau(u), \phi(e), \tau(v))$, SaG follows the Heterogeneous Graph Transformer formulation of Hu et al. [72] implemented via PyTorch Geometric’s HGTConv [82]. Entity-specific projections $W_{\tau(v)}$ first map raw features $x_v \in \mathbb{R}^{19-25}$ into the shared D -dimensional hidden space: $h_v^{(0)} = \text{LayerNorm}(\text{GELU}(W_{\tau(v)}x_v))$. Relational mutual attention across H heads incorporates type-parameterized Key ($K(u) = h_u^{(l-1)}W_K^{\tau(u)}$), Query ($Q(v) = \tilde{h}_v^{(l-1)}W_Q^{\tau(v)}$), and Value ($V(u) = h_u^{(l-1)}W_V^{\tau(u)}$) projections along with the edge representation $\tilde{h}_v = h_v + e'_{uv}$. Crucially, attention scores scale by a learned per-meta-relation prior $\mu_{\langle \tau(u), \phi(e), \tau(v) \rangle}$ (`p_rel` in PyG), which lets the model weight an entire relation triple up or down independently of individual node embeddings; this parameter directly captures the relational typing effect evaluated in Section 7.2. Message passing operates bidirectionally across both forward and transposed relation views (G_{analysis} and $G_{\text{analysis}}^\top$) to capture downstream starvation and upstream backpressure simultaneously, followed by residual aggregation, dropout ($p = 0.10$), and layer normalization across layers $l \in \{1, \dots, L\}$.

Training Protocol and Optimization Hyperparameters. Models are optimized end-to-end with AdamW ($\eta = 3 \times 10^{-4}$, weight decay 10^{-4} , post-attention dropout 0.10) under cosine decay with warm restarts ($T_0 = 75$, $T_{\text{mult}} = 2$, $\eta_{\text{min}} = 3 \times 10^{-6}$), for up to 300 epochs with early stopping at patience 30 on validation loss over labeled nodes. Inductive subgraphs are packed per scenario without mini-batch subsampling, and validation masks isolate held-out nodes. Five seeds {42, 123, 456, 789, 2024} are evaluated throughout, redrawing partitions and initializations for each. The architectural hyperparameters ($D = 64$, $H = 4$, dropout, learning rate, schedule) follow conventional HGT values [72]; the loss coefficients of Equation (5) were set by informed judgment, there being no convention for a five-term multi-task loss. Neither was tuned against the in-distribution test split or the LOSO folds, and no search was run over them. This avoids selection leakage but does not place either configuration near its own optimum: the comparison is between untuned configurations, and we state it as such.

4.2. Multi-Task Prediction Heads and Dimension Masking

From the final node embeddings $h_v^{(L)}$, SaG utilizes specialized multi-task prediction heads:

- **Reliability Head:** $\hat{R}(v) = \sigma(\text{MLP}_R(h_v)) \in [0, 1]$
- **Maintainability Head:** $\hat{M}(v) = \sigma(\text{MLP}_M(h_v)) \in [0, 1]$
- **Composite Failure Impact Head:** $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{R}(v) \parallel \hat{M}(v))) \in [0, 1]$
- **Relationship Criticality Head:** $\hat{Q}(u, v) = \sigma(\text{TypedEdgeEncoder}_{\phi(e)}(h_u, h_v, e_{uv})) \in [0, 1]$. Disabled throughout the evaluation reported here; every harness instantiates the model with edge prediction switched off, so $\hat{Q}(u, v)$ is neither trained nor scored. Described for completeness.

4.2.1. Optimization Objective and Dimension Masking

Under our headline experimental protocol, the active optimization objective directly balances cascade impact regression accuracy, auxiliary representation learning, global listwise ranking monotonicity, and fine-grained pairwise margin constraints:

$$\mathcal{L}_{\text{active}} = \text{MSE}(\hat{I}^*(v), I^*(v)) + 0.5 \cdot \text{MSE}(\hat{R}(v), I^*(v)) + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}} \quad (5)$$

where $I^*(v)$ is the simulated cascade impact defined by the primary oracle (Section 4.3), $\mathcal{L}_{\text{rank}}$ is the ListMLE listwise ranking loss [83] parameterized by temperature τ :

$$\mathcal{L}_{\text{rank}} = -\frac{1}{N} \sum_{i=1}^N \left(\frac{\hat{s}_{\pi_i}}{\tau} - \log \sum_{j=i}^N \exp \left(\frac{\hat{s}_{\pi_j}}{\tau} \right) \right) \quad (6)$$

where $\pi = (\pi_1, \dots, \pi_N)$ denotes the permutation of nodes sorted in descending order of ground-truth impact $I^*(v)$, and $\hat{s}_v = \hat{I}^*(v)$. At the baseline default $\tau = 1.0$, the formulation reduces to standard ListMLE; the temperature parameter $\tau < 1.0$ is a configurable hyperparameter that sharpens probability distributions over narrow prediction margins. Pairwise ordering fidelity is guided by margin-ranking loss $\mathcal{L}_{\text{pairwise}} = \frac{1}{|P|} \sum_{(u,v) \in P} \max(0, \gamma - (\hat{s}_u - \hat{s}_v))$ with margin $\gamma = 0.05$ over pairs $P = \{(u, v) \mid I^*(u) - I^*(v) > \gamma\}$. Permutation-level ListMLE ($\mathcal{L}_{\text{rank}}$) provides the primary gradient force for global rank monotonicity (ρ), while pairwise margin ($\mathcal{L}_{\text{pairwise}}$) penalizes small-margin inversions among adjacent components.

General Architectural Envelope and Dimension Masking: This active loss is the operational instance of a generalized multi-task framework:

$$\mathcal{L} = \mathcal{L}_{\text{composite}} + 0.5 \cdot \mathcal{L}_{\text{dimension}} + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}} + \lambda_{\text{RM}} \cdot \mathcal{L}_{\text{consistency}} \quad (7)$$

where $\mathcal{L}_{\text{dimension}} = \frac{1}{\sum_d m_d} \sum_{d \in \{R, M\}} m_d \cdot \text{MSE}(\hat{d}(v), d^*(v))$ uses a boolean dimension mask $m = [m_R, m_M]$, and $\mathcal{L}_{\text{consistency}} = \text{MSE}([\hat{R}(v), \hat{M}(v)]_{v \in \text{unlabeled}}, [R_{\text{RM}}(v), M_{\text{RM}}(v)]_{v \in \text{unlabeled}})$ provides optional semi-supervised alignment with the explanation layer on unlabeled nodes.

In our evaluation, two principled constraints govern this general envelope:

1. **Strict Pathway Independence:** We enforce $\lambda_{\text{RM}} = 0$ throughout all training sweeps, guaranteeing that the predictive GNN and the diagnostic explanation layer (Section 5) share zero parameters and remain strictly decoupled.
2. **Dimension Masking for Unobserved Maintainability:** Because dynamic cascade simulation ($I^*(v)$) observes runtime failure reachability rather than source-code maintainability, maintainability ground truth is unobserved during dynamic simulation. Setting $m = [1, 0]$ ensures the unobserved maintainability head is not artificially penalized or driven toward zero during backpropagation.

Consequently, setting $m_M = 0$ and assigning the cascade reachability target to both supervised heads ($R^*(v) \equiv I^*(v)$) causes $\mathcal{L}$ to reduce exactly to $\mathcal{L}_{\text{active}}$ (Eq. (5)). Here, $\hat{R}(v)$ re-enters the composite head as an input ($\hat{I}^* = \sigma(\text{MLP}_C(h_v \parallel \hat{R} \parallel \hat{M}))$), functioning as a feature-enrichment auxiliary representation rather than independent multi-task supervision. We report this objective as implemented to avoid any pretense of multi-dimensional ground truth.

4.2.2. Domain-Reweight Criticality

ISO/IEC 25019’s Context of Use specifies that the relative weighting of reliability and maintainability is determined by deployment requirements rather than being fixed. The framework delivers a reweighting $\hat{Q}_{\text{domain}}(v) = q_R \hat{R}(v) + q_M M_{\text{static}}(v)$ to capture this flexibility. Because maintainability is unobserved during dynamic simulation ($m = [1, 0]$), headline results report $\hat{I}^*(v)$ directly; sensitivity relative to the static RM baseline is evaluated in Section 7.3.

4.3. Ground-Truth Simulation Oracles

To evaluate predictive accuracy before deployment without relying on production runtime telemetry, SaG executes discrete-event failure simulations over the raw structural multigraph $G_{\text{structural}}$. Table 3 summarizes the four component-level oracles and their distinct roles across the evaluation program.

- **Cascade Reachability Oracle ($I^*(v)$),** evaluated via discrete-event cascade fault injection: crashes component v , propagates outages across dependent topics, brokers, and links by breadth-first traversal, and returns the mean fractional feed loss over the subscriber population, each subscriber contributing the unweighted mean loss of the topics it subscribes to. A topic’s feed loss is the fraction of its publishers that have failed (for a topic with no publisher, the fraction of its failed routers), scaled by a QoS ladder

Table 3: Simulation oracles, operational constructs, and evaluation roles.

Oracle	Physical Mechanism	Nature	Role in Evaluation
$I^*(v)$	BFS cascade reachability + QoS ladder	Deterministic	Primary ranking target (RQ1, RQ2, RQ4)
$I_{\text{comp}}(v)$	Severity mixture: reachability + fragmentation	Deterministic	Explanation layer / Validate gate
$I_{\text{dyn}}(v)$	Discrete-event SimPy message queuing	Stochastic	Convergent-validity probe (RQ3)
$I_M(v)$	Reverse DEPENDS_ON traversal	Deterministic	Unsupervised maintainability reference

($\times 1.2$ RELIABLE, $\times 1.15$ high/urgent priority, $\times 1.05$ medium) and clamped to $[0, 1]$. The denominator is the full subscriber set of the intact graph, so subscribers that themselves fail are retained in the average rather than excluded. The implementation admits a per-publisher rate weighting, but rates are declared per topic throughout our corpus, so it reduces exactly to this publisher fraction. This is the **primary continuous target label** throughout.

How much QoS is in this label. The ladder reads reliability and transport priority only; durability never enters I^* , despite carrying the largest of the three elicited sub-weights (0.62; Section 3.2). That omission does not bound the label’s QoS content, because disabling QoS scaling entirely leaves the Application ordering nearly intact — mean Spearman $\rho = 0.965$ against the ladder across the twelve folds (range 0.891–0.999) — and substituting a durability-aware $w(t)$ scaling moves it less still ($\rho = 0.977$). The top- K set is the sensitive construct: ladder and topology-only labels agree at mean Jaccard 0.678, so QoS changes *which* components are named critical without changing their order. I^* is therefore a near-topological target carrying a QoS term at its threshold boundaries, which bounds what any QoS-encoding result can be crediting (Section 7.3.1).

- **Multi-Metric Composite Oracle** ($I_{\text{comp}}(v)$), evaluated via multi-metric failure simulation: a severity-weighted mixture of reachability loss, fragmentation, throughput loss and flow disruption, with AHP-derived coefficients (0.35, 0.25, 0.25, 0.15). Those coefficients come from a rank-one comparison matrix, so they record their origin without independently justifying them. They are not swept in our sensitivity analysis — a gap worth naming because I_{comp} supplies the labels for the explanation layer’s evaluation. It is reserved for Validate-stage gates and prescriptive verification, never for forecasting ranking.
- **Dynamic Queue-Flow Oracle** ($I_{\text{dyn}}(v)$), evaluated via discrete-event message-flow queue simulation (built on SimPy [84]): simulates emission rates, stochastic latencies, broker buffer saturation, and queue drops under fault injection, extracting the drop in delivered message rate to surviving consumers. The engine instruments Google SRE’s *Four Golden Signals* across pre- and post-fault windows — latency decomposed into queue wait and service time at p50/p95/p99, traffic in Hz and KB/s, errors spanning deadline violations and overflow discards, and time-weighted mean queue depth for saturation — as diagnostic telemetry. They are kept out of $I_{\text{dyn}}(v)$ rather than mixed into it, because crashing a high-rate publisher clears downstream subscriber queues (*contention relief*, $\rho = -0.499$ between delivery loss and tail latency delta). An additive composite would mathematically cancel delivery damage with latency reduction. $I_{\text{dyn}}(v)$ is therefore kept strictly 1-dimensional, serving as an independent convergent-validity probe (Section 7.3.2).
- **Change-Propagation Oracle** ($I_M(v)$), evaluated via structural change-propagation analysis: a deterministic reverse-dependency traversal over the transpose of the six-rule DEPENDS_ON projection, blending change reach, weighted change impact, and normalized depth. It is a structural maintainability reference and is never used as a training label, which would make the supervision circular.

Topic Criticality Label Masking. The multi-metric failure simulator can incorporate declared topic criticality into its severity term. This feature is disabled, however, because topic criticality is a GNN input feature: using it would result in the predictor being measured against a transformation of its own input.

Primary Oracle Declaration and Role Assignment. The three reliability-facing oracles measure separate constructs, so we designate $I^*(v)$ the **primary continuous target** for all predictive ranking results (Tables 6–7, RQ1–RQ3): it has zero seed-to-seed variance, which is what deterministic CI/CD gating requires, whereas I_{dyn} adds stochastic latencies, bursty arrivals and synthetic buffer limits. That choice has a cost, since it makes the target a topological functional of the predictors’ own input (Section 4.4). $I_{\text{comp}}(v)$ is reserved for Validate-stage gates and prescriptive verification, $I_{\text{dyn}}(v)$ is an independent convergent-validity

probe (Section 7.3.2), and $I_M(v)$ a structural maintainability reference.

Cross-Oracle Convergent Validity. As detailed in Section 7.3.2, the three reliability oracles show substantial but sub-ceiling agreement on Applications ($\rho = 0.627$ for (I_{dyn}, I^*) against a 0.811–1.000 label noise floor), confirming distinct constructs. Consequently, results established against one oracle are never transferred to another; every evaluation explicitly references its underlying simulation oracle.

4.4. Input–Label Independence Guarantee

To prevent data leakage, SaG applies strict architectural separation. Feature Space is constructed exclusively from G_{analysis} using static structural topology, static code metrics, and declared QoS contracts. Label Space is evaluated exclusively on raw $G_{\text{structural}}$ through independent simulation oracles (cascade reachability injection, composite failure simulation, and dynamic message-flow simulation). No simulation outputs, failure trace histories, or dynamic execution telemetry are exposed as input attributes to the GNN or the explanation layer.

What this guarantee does and does not establish

The separation rules out circular feature construction: no predictor can read a transformation of the quantity it is scored against. It does not establish probabilistic independence between features and labels, and we do not claim it does. G_{analysis} is a deterministic projection of $G_{\text{structural}}$ (Section 3.2). Hence, the labels are — up to the simulator seed — a deterministic function of the same topology from which the features are computed. Two consequences follow, and both bound the results of Section 7.

First, $I^*(v)$ is a topological functional, defined as a breadth-first reachability computation over $G_{\text{structural}}$ scaled by a QoS ladder. The predictive task is therefore to recover a closed-form graph function from features derived from the same graph. Because the 18-dimensional feature block provides pre-computed global centrality indicators, the GNN’s role is synthesizing these positional and structural cues across heterogeneous relations rather than discovering global topology from scratch. As a result, an unparameterized centrality score is expected to perform competitively with a trained model (Section 7.1). The observed parity is an anticipated outcome of the experimental design rather than an unexpected limitation of graph learning. We state this explicitly to guarantee clarity.

Second, and more restrictively, no result in this paper is validated against an observed failure. Every label — on synthetic topologies and on the five open-source systems alike — is simulator-derived. The evaluation can establish whether a learned model recovers a simulator’s ordering on architectures it was not trained on. Whether that ordering corresponds to which components actually fail in production is a question this design cannot answer, and Section 8.3 treats it as the study’s principal construct-validity threat.

5. The Explanation Layer: Standards-Grounded Criticality Attribution

The predictor of Section 4 locates risk but says nothing about remedy. A component may be critical because it is an unreplicated single point of failure, an error-propagating cascade hub, or a high-coupling maintainability bottleneck, and each calls for a distinct intervention: replication, circuit breaker insertion, or architectural decoupling. This section formalizes the explanation layer that attributes those structural causes, decomposing criticality into a standards-grounded quality profile computed from the same typed node properties (Section 3.4) without sharing parameters with the predictor, and reaching flagged components through triage rather than data flow (Figure 1).

What this section is, and is not. This layer functions strictly as a diagnostic attribution instrument and design pattern for post-ranking triage, rather than an empirical ranking model. It is deliberately omitted from the primary predictive contributions in Section 1.5. Three empirical results bound its scope: (1) its standalone ranking correlation is modest ($\rho = 0.205$, trailing unweighted topological centrality on every fold, Section 7.1); (2) its elicited AHP weights are anti-predictive relative to an uninformative uniform prior ($\rho = 0.200$ vs. 0.319, Section 7.3); and (3) no human-subject study has yet measured developer efficiency under its guidance. What it provides is a rigorous, audited mapping from graph topology metrics to standardized ISO/IEC 25010/25019 quality sub-characteristics, providing actionable diagnostic rationales to guide refactoring.

5.1. Grounding in ISO/IEC Standards

In accordance with ISO/IEC 25010:2023 [13] and ISO/IEC 25019:2023 [14], SaG formalizes two primary criticality dimensions. Component Criticality (D_1) is service loss upon component failure; Relationship Criticality (D_2) is service decline upon channel severance.

Criticality is assessed across two orthogonal characteristics: **Reliability** (R) and **Maintainability** (M). Reliability is divided into **Fault Tolerance** (FT) and **Availability** (A). FT uses Reverse PageRank, in-degree, and cascade depth potential to inform redundancy and circuit breaker strategies. A uses directed articulation points, bridge ratios, and connectivity degradation to inform replication strategies. Maintainability (M) assesses structural coupling and code-level complexity, using betweenness, QoS-weighted fan-out, code quality penalties, and clustering to guide decoupling and refactoring. This partition maps each ISO/IEC sub-characteristic to its graph metrics and remediation roles. Safety and security considerations that require specialized hazard logs are excluded from purely structural topology analysis.

5.2. Composite Quality Score Formulation

All raw metrics are rank-normalized to the interval $[0, 1]$ within the graph. Quality sub-characteristics are formulated hierarchically using the Analytic Hierarchy Process (AHP) [63]:

- **Fault Tolerance** ($FT(v)$): Evaluates error cascade potential on transpose graph $G_{\text{analysis}}^\top$: $FT(v) = 0.45 \cdot \text{RPR}(v) + 0.30 \cdot \text{Deg}_{\text{in}}(v) + 0.25 \cdot \text{CDPot}_{\text{enh}}(v)$, where $\text{RPR}(v)$ is Reverse PageRank, $\text{Deg}_{\text{in}}(v) = d_{\text{in}}(v)/(|V| - 1)$ is normalized in-degree, and $\text{CDPot}_{\text{enh}}(v) = \text{depth}(v)/\max_{u \in V} \text{depth}(u)$ is normalized cascade depth potential on $G_{\text{analysis}}^\top$.
- **Availability** ($A(v)$): Identifies structural single points of failure across five terms: $A(v) = 0.25 \cdot \text{AP}_c^{\text{dir}}(v) + 0.20 \cdot \text{QSPOF}(v) + 0.20 \cdot \text{BR}(v) + 0.25 \cdot \text{CDI}(v) + 0.10 \cdot w(v)$, where $\text{AP}_c^{\text{dir}}(v)$ is Directed Articulation Point severity, $\text{QSPOF}(v)$ is QoS-weighted SPOF severity, $\text{BR}(v)$ is Bridge Ratio, $\text{CDI}(v)$ is Connectivity Degradation Index, and $w(v)$ is intrinsic QoS weight.
- **Reliability** ($R(v)$): Blends Fault Tolerance and Availability: $R(v) = r_{\text{FT}} \cdot FT(v) + (1 - r_{\text{FT}}) \cdot A(v)$ with $r_{\text{FT}} = 0.36$. Intra-dimension weights apply $\lambda = 0.70$ shrinkage blending with a uniform prior. Three of the framework's five comparison matrices (Impact, Maintainability, Availability) are rank-one by construction, so their Consistency Ratios are uninformative; the remaining two (Topic QoS and Fault Tolerance) carry genuine second-eigenvalue spread and their CR figures do mean what CR normally means. Supplementary Section S4 separates them. Elicited AHP weights rank worse than a uniform prior against dynamic simulation (Section 7.3), and we recommend the uniform prior. The shipped default nonetheless remains $\lambda = 0.70$, because the same constant parameterises the I_{comp} severity weights: changing it would re-label the composite oracle and with it every figure scored against that oracle. Decoupling the two is a prerequisite for changing the default, not a reason to defend it.
- **Maintainability** ($M(v)$): Blends structural coupling with static code analysis: $M(v) = 0.35 \cdot \text{BT}(v) + 0.30 \cdot w_{\text{out}}(v) + 0.15 \cdot \text{CQP}(v) + 0.12 \cdot \text{CouplingRisk}_{\text{enh}}(v) + 0.08 \cdot (1 - \text{CC}(v))$, where $\text{BT}(v)$ is Betweenness Centrality, $w_{\text{out}}(v)$ is QoS-weighted efferent coupling, $\text{CQP}(v)$ is Code Quality Penalty, and $\text{CC}(v)$ is local Clustering Coefficient.

The baseline composite quality score integrates both dimensions as follows: $Q(v) = 0.80 \cdot R(v) + 0.20 \cdot M(v)$. When evaluated under an ISO/IEC 25019 Context of Use vector $\vec{\omega} = [q_R, q_M]^\top$, the score is dynamically reweighted: $Q_{\text{domain}}(v) = q_R \cdot R(v) + q_M \cdot M_{\text{static}}(v)$. Components are partitioned into Tukey tiers: CRITICAL ($Q > Q_3 + 1.5 \cdot \text{IQR}$), HIGH, MEDIUM, and MINIMAL. Across the benchmark topologies, this conservative Tukey upper fence flags an empirical mean of 4.2% of components (range 1.8%–8.3%), deliberately isolating the extreme right tail of architectural risk to prioritize developer intervention. High Availability (A) combined with low Fault Tolerance (FT) indicates a single point of failure that necessitates replication. In contrast, high Fault Tolerance (FT) identifies an error-cascade hub that requires circuit breakers (Section 8.4). Supplementary Table S16 shows a representative Diagnostic Remediation Card, illustrating how the sub-characteristics translate topological metrics into refactoring actions.

5.3. Prescriptive Remediation and Counterfactual Verification

After attributing root causes, candidate repairs (broker replication, circuit breaker insertion, topic decoupling) are generated and counterfactually verified in memory. Mutations are accepted only if they reduce systemic impact beyond simulation seed noise ($\Delta \bar{I}_{\text{comp}} > \kappa \cdot \sigma_{\text{seed}}$, $\kappa \geq 1.0$) without introducing new articulation points. This counterfactual verification loop illustrates the architectural pattern linking diagnosis to remediation; formal developer studies and automated patch synthesis benchmarks are reserved for future work.

6. Experimental Setup

6.1. Datasets and System Corpus

The evaluation corpus comprises 2,812 components across seventeen system architectures, including twelve synthetic topologies that form the inductive cross-validation folds and five real-world reference systems withheld from all training procedures, as detailed in Table 4. The synthetic scenarios span diverse operational domains (autonomous vehicles, financial trading, healthcare integration, industrial SCADA, smart-city IoT, telecom RAN, cloud microservices, and enterprise application integration via centralized broker hubs/ESB; detailed in Supplementary Table S12).

Table 4: Overview of the evaluation corpus. The twelve synthetic topologies correspond to the inductive Leave-One-Scenario-Out folds described in Table 6, and the five real-world systems are excluded from all training folds and used exclusively for zero-shot transfer (Section 7.4). Per-scenario entity and edge counts are obtained from the committed topology files and verified through continuous integration.

Dataset	$ V $	$ V_{\text{app}} $	Topics	Brokers	Hosts	Libs	$ E $
Synthetic evaluation scenarios (11)	2,387	1,295	588	60	194	250	10,657
Synthetic case study — ATM (1)	74	26	27	5	8	8	261
Synthetic subtotal (12 LOSO folds)	2,461	1,321	615	65	202	258	10,918
Real-world subtotal (5 systems)	351	141	120	16	32	42	700
Total	2,812	1,462	735	81	234	300	11,618

6.1.1. Reproducibility of the Corpus

The benchmark corpus is designed to be fully regenerable rather than statically archived. Each dataset is deterministically generated from its configuration file via `python cli/generate_graph.py batch -input-dir data/scenarios -output-dir <path>`. A companion manifest records the random seed, entity counts, git commit hash, and a SHA-256 cryptographic digest for each emitted topology. Continuous integration regression tests verify that every committed dataset regenerates byte-identically from its configuration and that all disk digests match the manifest. This procedure makes sure that third parties can reproduce the exact graphs used in these experiments, rather than sampling from similar distributions.

6.2. Baselines and Evaluated Predictors

Four primary predictor configurations, drawn from three distinct families, are evaluated alongside reference baselines. Table 5 provides a structured taxonomy of the evaluated predictors, their underlying graph representations, edge feature encodings, and empirical roles. Predictor names indicate the model family and substrate: an `-N` infix denotes a model trained on the complete native multigraph, its absence denotes the obtained Application–Library flow projection, and a `-QoS` suffix denotes a configuration that consumes declared QoS contracts. SaG throughout denotes the overall framework, not an individual predictor.

Table 5 summarizes the evaluated predictors, graph representations, and empirical roles across three families: heterogeneous graph learning (HGT-QoS with 16-D QoS edge vectors, and its ablation HGT), homogeneous graph learning (GAT-N-QoS with scalar $w(e)$, and unweighted GAT-N), and training-free structural baselines (Topo-QoS and unweighted Topo on the DEPENDS_ON flow projection). The `-N` infix denotes the native multigraph substrate; on flow projections, homogeneous variants are denoted GAT/GAT-QoS (Supplementary Table S15). The out-of-distribution evaluation (Table 6) additionally reports **RM** ($Q(v)$, Section 5) as a diagnostic reference baseline. Deterministic RM scoring also drives sensitivity sweeps in Section 7.3.

Table 5: Taxonomy of evaluated predictors and reference baselines. The **-N** infix denotes execution on the complete native multigraph; **-QoS** indicates inclusion of middleware QoS attributes.

Predictor	Evaluation Substrate	Typing	Edge Features	Parameters	Trained?	Empirical Role
Topo	G_{analysis} (Flow Projection)	No	None	0	No	Unweighted topological baseline
Topo-QoS	G_{analysis} (Flow Projection)	No	Scalar $w(e)$	0	No	QoS-weighted closed-form benchmark
GAT	G_{analysis} (Flow Projection)	Homogeneous	None	28,168	Yes	In-distribution counterpart of GAT-N (Supp. Table S15)
GAT-QoS	G_{analysis} (Flow Projection)	Homogeneous	Scalar $w(e)$	28,168	Yes	In-distribution counterpart of GAT-N-QoS (Supp. Table S15)
GAT-N	Native Multigraph	Homogeneous	None	28,168	Yes	Untyped, unweighted GNN floor
GAT-N-QoS	Native Multigraph	Homogeneous	Scalar $w(e)$	28,168	Yes	Isolates QoS channel without typing
HGT	Native Multigraph	Heterogeneous	Relation 1-hot	434,620	Yes	Isolates relational typing without QoS
HGT-QoS	Native Multigraph	Heterogeneous	16-D QoS Vector	434,620	Yes	Proposed full learned model
RM / $Q(v)$	G_{analysis} (Analysis Graph)	Per-type rules	Scalar $w(e)$	0	No	Diagnostic attribution reference

6.2.1. Evaluation Substrates and Parity Guarantee

Predictors in this study operate on matched substrates within their respective families:

Graph Learning Models (GAT-N-QoS, HGT-QoS): Under Leave-One-Scenario-Out (Table 6), both learned predictors ingest the complete native typed multigraph across all five entity types (recorded using the shared **-N** infix). In-distribution (Supplementary Table S15), GAT/GAT-QoS consume the Application-Library **DEPENDS_ON** projection, confounding typing with multi-entity visibility. GAT-N-QoS uses per-type projection with untyped GATConv across edges, whereas HGT-QoS uses relation-specific HGTCov weights. The edge channel also differs: GAT-N-QoS consumes a scalar $w(e)$, while HGT-QoS consumes all 16 dimensions. The parameter budget (434,620 vs. 28,168) and message directionality (103,725 reverse parameters) also remain open confounds: four control arms (GAT-N-C, GAT-N-QoS-C, GAT-N-QoS16-C, and HGT-QoS-U) are formally specified in Amendment 2 of our registered analysis plan to isolate these factors in future sweeps (Section 8.4).

Training-Free Structural Baselines (Topo, Topo-QoS): We evaluate topological baselines on the obtained Application-Library **DEPENDS_ON** projection (Section 3.2), since raw multigraphs route messages via topics/brokers, leaving Application nodes with near-zero betweenness. Because these two baselines carry every headline contrast in Section 7, we state them in closed form rather than by name. Both are convex combinations of a path-traversal term and a cut-vertex term:

$$\text{Topo}(v) = 0.6 \cdot \text{BT}(v) + 0.4 \cdot \text{AP}(v), \quad \text{Topo-QoS}(v) = 0.6 \cdot \text{BT}_w(v) + 0.4 \cdot \text{AP}(v) \quad (8)$$

where $\text{BT}(v)$ is normalized betweenness centrality on the projection, $\text{AP}(v) \in \{0, 1\}$ indicates whether v is an articulation point of the projection’s undirected form, and $\text{BT}_w(v)$ is betweenness computed over edge distances $d(e) = 1/(w(e) + \epsilon)$ with $\epsilon = 10^{-6}$, so that strongly coupled edges are short and attract shortest paths. The projection itself carries two of the six rules of Table 2: Rule 1 joins a subscriber to each publisher of a topic it consumes, at $w = 1 - \prod_{t \in T} (1 - w(t))$, and Rule 5 joins an application to each library it uses, at the median topic weight.

Two properties of this pair matter for how Section 7 should be read. First, *only the traversal term is QoS-weighted*: the articulation term is identical in both, so the entire **Topo-QoS** margin is carried by re-weighting shortest paths. Second, the weighting *degenerates gracefully*: when no edge of a graph carries a non-unit weight, BT_w reduces to BT and **Topo-QoS** coincides with **Topo** on that graph. The 0.6/0.4 split is a declared convention, not a fitted parameter, and is not tuned per scenario.

Ground-Truth Independence Guarantee: Crucially, **no predictor in either group accesses $G_{\text{structural}}$** : that raw topology is strictly reserved for simulation oracles (Section 4.4), verified by `tests/test_independence_guarantee.py`. Regardless of substrate, all variants are scored on the same independently resolved Application node set (V_{app} , Section 6.3).

6.3. Evaluation Metrics and Protocols

Figure accessibility. All figures employ the Okabe-Ito palette with distinct markers and hatchings to ensure monochrome legibility.

Ranking Precision: Evaluated via Spearman ρ against the simulated impact $I^*(v)$ from the primary oracle (Section 4.3). **Critical-Set Identification:** Measured via $F_1@K$ for top- K components ($K = \text{round}(0.20 \cdot |V_{\text{app}}|)$). Because both the predicted and reference sets have exactly K members, precision, recall and $F_1@K$ coincide identically: the column is top- K set overlap, not an F_1 score, and it is reported under that reading everywhere it appears. Identification at operating points where precision and recall

are free to differ — $F_1@T$ against the labels’ own critical set, threshold-free PR-AUC, and rank-weighted nDCG@10 — is reported separately in Supplementary Table S13, because those quantities answer a question $F_1@K$ structurally cannot. **Statistical Significance:** Paired Wilcoxon signed-rank tests [85] ($p < 0.05$) and bootstrap 95% CIs ($B = 2,000$) over folds [86, 87]. In the 12-fold LOSO design, the power floor is $p = 0.00049$.

One confirmatory family, and everything else exploratory. The confirmatory family is the pre-registered one and contains two contrasts: HGT-QoS against Topo-QoS, and HGT against Topo-QoS, Holm-corrected across those two alone. Neither reaches $\alpha = 0.05$ (Section 7.1). Every other contrast in this paper — the remaining full-population comparisons against Topo-QoS, the three orthogonal quantities of the 2×2 (Table 8), and the four simple effects — was formulated after the results existed and is reported as exploratory. Earlier versions of this manuscript corrected two overlapping families, which made the same contrast survive under one correction and not the other; the 2×2 quantities are still Holm-corrected within their own block, and that block is labelled post-hoc where it appears. For orientation, the largest exploratory effects are Topo-QoS over Topo (+0.204, 12/12, $p = 0.0005$) and unweighted typing, HGT over GAT-N (+0.234, 12/12, $p = 0.0005$), while the two contrasts that carry the QoS channel into an already-typed model do not separate: typing with the QoS channel present reaches $p = 0.1294$ (9/12 folds) and the QoS edge ablation under typing $p = 0.2036$ (10/12).

Registered analysis plan. The primary out-of-distribution contrast (HGT-QoS vs. Topo-QoS under LOSO, 5 fixed seeds, fold as unit of analysis) was registered in the replication package, with its protocol, statistic, unit of analysis and reporting commitment fixed, before the revised harness produced any result. We describe it as registered rather than pre-registered, and the distinction is not cosmetic: the plan is a file in our own repository with no third-party timestamp, and a prior eight-fold run of the same contrast — since withdrawn, because it reproduced from no commit — predates it and is what motivated writing the plan down. What the registration establishes is that the analysis was not selected after seeing the twelve-fold result; what it cannot establish is that the question was asked in ignorance of any earlier estimate. As reported in Section 7.1, the margin did not reach statistical significance, which is the outcome the plan committed to reporting.

6.3.1. Evaluation Population and Protocols

Each predictor within an evaluation table is scored on an identical node population, resolved strictly from scenario topology and ground truth, specifically the **Application** set (V_{app}) unless otherwise noted. Pooling node types conflates distinct base rates and can trigger Simpson’s paradox (Section 7.3).

In-Distribution Evaluation: Stratified 60% train / 20% val / 20% test node splits over five seeds {42, 123, 456, 789, 2024}, redrawing partitions and initializations. **Inductive LOSO Cross-Validation:** Models are trained on eleven scenarios and test zero-shot on the held-out twelfth across all 12 folds under equal 3-layer depth and inner-split early stopping (Section 8.4). **Real-World Architectural Transfer:** Synthetic-trained models are evaluated zero-shot on five open-source systems without fine-tuning.

7. Results and Empirical Analysis

Empirical results for RQ1–RQ5 are presented across the twelve-fold inductive benchmark and five authentic open-source distributed systems. The evaluated populations are strictly stratified on the Application service set (V_{app}) under the input–label independence guarantee (Section 4.4).

7.1. RQ1: Graph Learning vs. Structural Baselines

RQ1 Summary (Predictive Efficacy): **HGT-QoS** ($\rho = 0.638$) achieves highest numerical correlation but does not significantly outperform the unparameterized **Topo-QoS** baseline ($\rho = 0.553$, $\Delta\rho = +0.085$, $p = 0.151$, 95% CI spanning zero). Because ground-truth reachability $I^*(v)$ is a topological functional, closed-form QoS-weighted centrality captures broad failure propagation without training. Restricting evaluation to active components roughly halves correlation for every predictor, learned or not ($\rho_{>0} \in [0.102, 0.356]$ against $\rho \in [0.205, 0.638]$), showing that full-population correlation largely reflects separation of inert sink nodes.

What the in-distribution results can and cannot support. Per-scenario in-distribution figures are reported in Supplementary Table S15 rather than here, because no comparison can be drawn down their columns: GAT/GAT-QoS consume the Application-Library projection while HGT/HGT-QoS consume the native multigraph (Section 6.2.1), so a difference between the families confounds message passing with multi-entity visibility. Within a predictor the cells are still informative. In Healthcare, Topo-QoS achieves $\rho = 0.399$ but fails at critical triage ($F_1 = 0.000$); in the synthetic Microservices fold, unaugmented HGT suffers degradation ($\rho = 0.141, F_1 = 0.300$), whereas HGT-QoS recovers both ranking monotonicity ($\rho = 0.664$) and top- K overlap ($F_1@K = 0.600$).

7.1.1. Out-of-Distribution (LOSO) Generalization

Inductive Leave-One-Scenario-Out cross-validation asks each model to predict cascading criticality on an entirely unseen topology, and the twelve folds are this paper’s primary anchor for generalization across architectural archetypes. Per-fold breakdowns are in the Supplementary Material (Table S12, Section S10); the main text reports the cross-fold summary (Table 6) and the active-stratum contrast (Table 7):

Table 6: Inductive LOSO evaluation, Application population, twelve folds. Learned variants share training set, the native multigraph substrate (hence the -N infix; Section 6.2), depth, and selection rule (Section 6.3), differing in typing and edge channel; parameter budget and message-passing directionality remain unmatched confounds that Table 8 controls for (Section 8.4). Fold score = mean over five seeds; **Fold** σ = spread of the twelve fold means; **Seed** σ = median within-fold spread (zero by construction for deterministic baselines); CI = bootstrap over folds ($B = 2,000$). $\Delta\rho$ is paired by fold against Topo-QoS, the pre-registered comparator (Section 6.3), with its own bootstrap interval; an interval spanning zero means the contrast is not resolved at twelve folds, which is the case for every learned variant.

Predictor / Reference	Mean LOSO ρ	95% CI	$\Delta\rho$ vs Topo-QoS	Fold σ	Seed σ	Critical-Set $F_1@K$	Requires Training
<i>Training-free structural baselines</i>							
Topo	0.349	[0.254, 0.452]	-0.204 [-0.286, -0.122]	0.173	—	0.366	No
Topo-QoS	0.553	[0.443, 0.657]	— (reference)	0.192	—	0.388	No
<i>Learned predictors (shared native substrate, matched training set)</i>							
GAT-N	0.317	[0.254, 0.381]	-0.236 [-0.342, -0.125]	0.111	0.298	0.328	Yes
GAT-N-QoS	0.604	[0.538, 0.665]	+0.051 [-0.067, +0.169]	0.112	0.024	0.431	Yes
HGT	0.551	[0.474, 0.617]	-0.002 [-0.066, +0.072]	0.124	0.114	0.427	Yes
HGT-QoS	0.638	[0.561, 0.710]	+0.085 [-0.029, +0.194]	0.133	0.052	0.424	Yes
<i>Diagnostic reference — not a ranking model</i>							
RM / $Q(v)$	0.205	[0.092, 0.320]	-0.348 [-0.432, -0.265]	0.195	—	0.322	No

Each of the twelve folds holds out one scenario for zero-shot testing and trains on the remaining eleven, with all variants scored on the same Application node set (26 to 300 nodes, giving K between 5 and 60) and paired Wilcoxon tests over folds. For $F_1@K$, HGT-QoS achieved 0.424 compared to Topo-QoS’s 0.388 ($\Delta = +0.037$, prevailing in 7 of 12 folds, $W = 29.0, p = 0.470$). The untyped GAT-N-QoS attained 0.431 ($\Delta = -0.006$, prevailing in 5 of 12 folds, 1 tie, $W = 27.5, p = 0.653$), indicating that critical-set identification does not statistically distinguish the typed model from either untyped learning or the QoS-weighted baseline. **Label-noise ceiling.** No predictor can exceed the reproducibility of its own labels. Re-running the oracle across the five seeds gives a test-retest rank correlation of 0.811–1.000 (median 0.982), so HGT-QoS’s $\rho = 0.638$ recovers roughly 65% of the attainable signal. Top- K sets are far noisier — cross-seed Jaccard median 0.847, falling to 0.370 on Logistics Fleet — which is why the $F_1@K$ margins are less stable than the ranking ones. The least reproducible fold (Microservices, 0.811) is not one the typed model loses; on this corpus the low-ceiling folds and the lost folds are disjoint.

Key Insights concerning RQ1:

1. **Typed learning is the best configuration, but not demonstrably better than the QoS baseline.** HGT-QoS leads all predictors out-of-distribution ($\rho = 0.638$). Against training-free Topo-QoS it is +0.085 (9/12, $W = 20.0, p = 0.151$, CI [-0.029, +0.194]), an interval that includes zero; unaugmented HGT is indistinguishable from the baseline outright (-0.002, 3/12, $p = 0.470$). Neither pre-registered contrast reaches significance, and we report that as the answer rather than as a near miss.
2. **A QoS-weighted structural score is a genuinely strong baseline — and untyped learning is worse than it.** Topo-QoS reaches $\rho = 0.553$ zero-shot, beating unweighted Topo on all twelve folds (+0.204, $p = 0.0005$). More pointedly, the untyped, unweighted learned model loses to it decisively

(GAT-N, -0.236 , $2/12$, $p = 0.0024$). On this task, a homogeneous graph network trained on eleven architectures does not reach what a closed-form centrality score achieves with no training at all. Any claim that graph learning is required must be made against this baseline.

3. **Critical-set identification does not favor the typed model.** On $F_1@K$, HGT-QoS scores 0.424 against GAT-N-QoS's 0.431 and HGT's 0.427 — a three-way tie within noise — while all three numerically lead Topo-QoS (0.388), though without statistical significance throughout folds ($\Delta = +0.037$, $p = 0.470$).
4. **Power is not the limiting factor.** At $n = 12$, the design tolerates four lost folds and still reaches $\alpha = 0.05$, provided the losses are smallest in magnitude. HGT-QoS's are not: it loses Enterprise (-0.335) and Telecom RAN (-0.169) to Topo-QoS by the two largest margins in the set, which is what holds W at 20.0 . Enlarging the corpus will not resolve this; we must understand the inversions instead (Section 7.2.1).
5. **The explanation layer is weakly predictive, not noise.** RM/ $Q(v)$ reaches $\rho = 0.205$, losing to unweighted Topo on every fold (-0.144), so no ranking claim is made for it. Its interval $[0.092, 0.320]$ stays above zero, and it supplies interpretable diagnostics without training (Section 5). Table 6 lists it as a reference point, not a competitor.

7.1.2. The Active Stratum: Ranking Quality Separated From Inertness Detection

Between 21% (Microservices) and 52% (Healthcare) of each held-out Application population carries exactly zero simulated impact, so a predictor can score well by separating components that can propagate a failure from those that cannot, without ordering the propagating ones correctly. These are different capabilities, so we re-score all twelve folds on the *active stratum* — the $n_{>0}$ components with strictly positive impact — using the same predictions, folds and seeds. Table 7 reports both.

Table 7: LOSO means over twelve folds on the full Application population (ρ) and restricted to components with strictly positive ground-truth impact ($\rho_{>0}$). Same predictions, folds, and seeds; only the evaluated subset differs. Retained is defined as the active-stratum ratio $\rho_{>0}/\rho$ (percentage of full-population correlation preserved).

Predictor	ρ (full)	$\rho_{>0}$ (active)	Retained
RM / $Q(v)$	0.205	0.102	49%
Topo	0.349	0.181	52%
Topo-QoS	0.553	0.280	51%
GAT-N	0.317	0.159	50%
GAT-N-QoS	0.604	0.328	54%
HGT	0.551	0.299	54%
HGT-QoS	0.638	0.356	56%

Two consequences follow.

1. **The restriction costs every predictor roughly half its correlation, learned or not.** Retained fractions run from 49% (RM) to 56% (HGT-QoS) with no systematic separation between the training-free and learned families (Topo-QoS 51%, GAT-N 50%, HGT 54%). Roughly half of every predictor's full-population correlation reflects separating inert components from active ones, a property of the label distribution rather than a discriminator between methods.
2. **The method ordering is unchanged, and so are the verdicts.** On the active stratum HGT-QoS still leads ($\rho_{>0} = 0.356$), ahead of Topo-QoS (0.280) and GAT-N-QoS (0.328). Nothing in Section 7.1 or Section 7.2 turns on whether zero-impact components are included; we report both columns because they answer alternative operational questions.

7.2. RQ2: Value of Typed Heterogeneity

RQ2 Summary (Typing vs. QoS):. Relational typing and continuous QoS edge encodings act as empirical substitutes rather than complements. While each mechanism individually exhibits a significant main effect over unweighted baselines (+0.134 and +0.187, Holm $p = 0.0015$), their factorial interaction is sub-additive ($\Delta\rho = -0.199$, negative on all 12 folds, $p = 0.0005$). In pub-sub architectures, QoS contracts align with

communication relations; thus, edge features and relation-specific weights supply redundant traversal indicators, yielding diminishing returns when combined.

Whether relation typing improves prediction over homogeneous message passing depends entirely on the comparator it is ablated against:

- **Both factors have a real main effect.** Averaged over the other factor’s levels, relation typing is worth $\Delta\rho = +0.134$ (12/12 folds, $p = 0.0005$, Holm 0.0015) and the QoS edge channel +0.187 (11/12, Holm 0.0015), subject to the capacity and directionality confounds below.
- **But they interact, strongly and sub-additively.** The difference of differences — what typing buys with the QoS channel present, minus what it buys without — is -0.199 , negative on all twelve folds ($W = 0.0$, $p = 0.0005$, Holm 0.0015, CI $[-0.258, -0.147]$). The simple effects show it from both sides: typing is worth +0.234 without the QoS channel and +0.035 with it; the QoS channel +0.287 without typing and +0.087 with it.
- **In-distribution fitting is not evidence either way** (Supplementary Table S15): the homogeneous pair reads the Application–Library projection while the typed pair reads the native multigraph, confounding message passing with multi-entity visibility.

Typing and QoS encoding are substitutes, not complements. Each mechanism alone lifts the plain baseline from $\rho = 0.317$ to 0.55–0.60; together they reach 0.638, barely more than either achieves alone. Both supply identical information: which relation an edge traverses. Weight matrices encode it for typing; edge vectors encode it for QoS. Neither channel can track impact the oracle does not express, which bounds what either can be crediting (Section 7.3.1).

Reference arm and confounders. Both large simple effects are measured against GAT-N, an unweighted floor ($\rho = 0.317$, seed $\sigma = 0.298$). Substrate, training set, depth and early stopping are held constant, but parameter capacity (434,620 against 28,168, $15.4\times$) and message directionality (103,725 reverse-relation parameters in HGTCnv) are not, so the +0.134 typing margin reflects a joint architectural transition rather than typing alone. Four control arms that would separate these — GAT-N-C and GAT-N-QoS-C (capacity-matched untyped, 437,496 and 439,272 parameters), GAT-N-QoS16-C (edge-channel width) and HGT-QoS-U (unidirectional, 330,895) — are implemented and registered as Amendment 2 of the analysis plan, with a decision rule fixed before any control result exists, but none has been run at the reported budget. The confound therefore stands as stated, and on present evidence the $15.4\times$ capacity gap buys a margin that does not reach significance once the QoS channel is present.

Table 8: The 2×2 over relation typing (T) and the QoS edge channel (Q), whose four cells are the four reported learned arms: GAT-N ($\neg T \neg Q$), HGT (T-Q), GAT-N-QoS ($\neg T Q$), HGT-QoS (TQ). **Holm correction is applied across the three orthogonal quantities in the upper block only**; the four simple effects below are algebraically determined by those three and are reported descriptively (Section 7.3.1). Main effects average over the other factor’s levels; the typing effect reflects the joint transition to HGT and is confounded (Section 8.4). **Won** counts folds with $\Delta > 0$; the interaction is *negative* on all twelve. All quantities are post-hoc, and none were pre-registered. Fold overlap makes the reported p -values nominal (Section 8.3).

Quantity	Contrast	$\Delta\rho$	Won	W	p	p_{Holm}
<i>The 2×2: three orthogonal quantities, Holm-corrected across these three</i>						
Typing (main effect)	averaged over Q	+0.134	12/12	0.0	0.0005	0.0015
QoS channel (main effect)	averaged over T	+0.187	11/12	2.0	0.0015	0.0015
Typing $\times$ QoS interaction	difference of differences	-0.199	0/12	0.0	0.0005	0.0015
<i>Simple effects — descriptive, not separately corrected</i>						
Typing, QoS absent	HGT vs. GAT-N	+0.234	12/12	0.0	0.0005	—
Typing, QoS present	HGT-QoS vs. GAT-N-QoS	+0.035	9/12	19.0	0.1294	—
QoS channel, typing absent	GAT-N-QoS vs. GAT-N	+0.287	11/12	1.0	0.0010	—
QoS channel, typing present	HGT-QoS vs. HGT	+0.087	10/12	22.0	0.2036	—

The interaction is not an artefact of the correlation scale. A difference of differences computed on Spearman ρ is not scale-free. Rho is bounded on $[-1, 1]$ and compresses as it approaches either end, so two mechanisms that each move a predictor toward the attainable ceiling can appear to interact sub-additively

even when they contribute independently under any monotone rescaling. Because the substitution claim is exactly a claim about an interaction, we recomputed all three quantities with each fold’s ρ passed through arctanh first, which removes that compression. The interaction survives and grows: $\Delta z = -0.233$, negative on all twelve folds, $W = 0.0$, $p = 0.0005$ (Holm 0.0015), bootstrap 95% CI $[-0.309, -0.165]$, against -0.199 on the raw scale. Both main effects likewise survive ($+0.186$ typing, $+0.269$ QoS channel). Supplementary Table S14 reports the transformed block in full. Sub-additivity is therefore a property of the two mechanisms on this corpus, not of the metric used to measure them — which is what licenses reading them as substitutes, subject to the capacity and directionality confounds above.

7.2.1. Where Typed Learning Fails, and How It Can Be Detected

HGT-QoS loses to Topo-QoS on three folds, but only two of them are substantive: Enterprise ($\rho = 0.461$ vs. 0.795) and Telecom RAN (0.407 vs. 0.576). The third, AV System (0.722 vs. 0.753), is a near-tie at -0.030 and carries little interpretive weight. The two substantive inversions are what keep the RQ1 comparison below significance. All three losses fall where Topo-QoS is at its own strongest. Enterprise, AV and Telecom RAN are its 2nd, 3rd and 7th best folds of twelve (0.795 , 0.753 , 0.576), each at or above its mean of 0.553 . This supports reading the inversions as the learned model discarding structural signal the baseline retains, rather than as folds that are intrinsically hard — on the two synthetic folds where Topo-QoS is weakest (Microservices 0.265 , ATM 0.311) the typed model wins by $+0.229$ and $+0.210$.

Absence of a Label-Free Confidence Signature. We tested whether the dispersion $\hat{\sigma}$ of predicted scores over held-out applications could flag untrustworthy folds without labels. It cannot: the lowest-dispersion folds include ones with large positive margins, so no threshold on $\hat{\sigma}$ separates wins from losses. We report the direction of this result rather than its coefficients. The dispersion diagnostic was computed against a prediction export that the twelve-fold artifact behind Tables 6–8 supersedes, and no export exists at that fidelity, so the per-fold $\hat{\sigma}$ values are withheld pending a re-run and no coefficient is quoted for them anywhere in this paper. Graph size is measurable on the current artifact and also fails to flag difficulty in advance (rank correlation with margin -0.434 , $p = 0.159$), as does connection density.

Feature scale drift across scenarios remains the primary explanation for the Enterprise deficit, as Enterprise is the largest graph (520 nodes) and feature-scaling disparities are most acute in this case. Because the protocol strictly holds model hyperparameters constant across folds, there is currently no automated, label-free signal to predetermine whether an unseen architecture will favor the learned model or the training-free baseline (Section 8.4).

7.3. RQ3: Ablations and Sensitivity Analysis

RQ3 Summary (Ablations and Sensitivity): Only two of the explanation layer’s ten declared constants matter under Morris screening, and no configuration alters any comparison above. The elicited AHP weights are anti-predictive against a uniform prior, and the three reliability oracles agree substantially but below the label-noise ceiling.

This section presents ablations relevant to the primary claims, including the QoS edge encoding, cross-oracle agreement, and per-type stratification that informs the interpretation of the results that follow. Of the ten constants, only the AHP shrinkage λ and the Fault-Tolerance/Availability blend r_{FT} exhibit appreciable influence on ρ ($\mu^* = 0.134$ and 0.132 under Morris screening, compared to ≤ 0.025 for the remaining eight). No configuration of the topic-weight or QoS sub-weight constants alters any comparison reported above. The elicited AHP weights are, notably, *anti*-predictive: rank correlation decreases monotonically from 0.319 under a uniform setting to 0.200 under raw AHP judgment. We retain these weights because RM serves as an attribution instrument rather than a ranking model, a trade-off discussed in Section 8.4.

7.3.1. QoS Feature Ablation

To isolate the contribution of the continuous-categorical QoS edge features (Section 4.1.1) we evaluated **HGT**, an ablation whose edge features carry only scalar coupling and the relation one-hot. The result is not a second finding but Table 8 read from the other side: because the interaction is symmetric in its two factors, a conditional effect of typing on the QoS channel is necessarily a conditional effect of the QoS channel on

typing. *Without typing the encoding is decisive* — GAT-N-QoS 0.604 against GAT-N’s 0.317, +0.287, 11/12 folds, $p = 0.0010$, CI [+0.207, +0.365]. *With typing it is not significant* — HGT-QoS 0.638 against HGT’s 0.551, +0.087, 10/12, $p = 0.2036$. The significance of the *difference* between these rests on the interaction, not on the difference between their p -values. While preliminary subset sweeps suggested that QoS might offer an independent additive gain on top of typing, the full twelve-fold benchmark establishes that this marginal improvement does not reach statistical significance ($p = 0.2036$), demonstrating that typing and continuous QoS encodings function primarily as empirical substitutes.

The encodings also stabilise optimization, and there the asymmetry runs the other way: the median within-fold standard deviation over five seeds is 0.024 for GAT-N-QoS against 0.298 for GAT-N, and 0.052 for HGT-QoS against 0.114 for HGT. A model given neither channel is by far the least stable, and either one fixes it — consistent with both telling the model which relation an edge belongs to.

The target is nearly QoS-free, which bounds what either channel can credit.. These gains are earned against $I^*(v)$, whose ordering a topology-only relabeling recovers at mean $\rho = 0.965$ across the same twelve folds with no QoS term in the labeler (Section 4.3). The QoS edge channel cannot help the model track QoS-driven impact the oracle does not express, which is the strongest evidence we have for reading it as a relation-identity channel rather than a contract-semantics one. The label does move under QoS, but only at its top- K boundary (Jaccard 0.678), not in its ranking. A corpus whose oracle expressed QoS-driven impact in its *ordering* — deadline misses, durability replay, priority inversion — would be a stronger test of the encodings than the one we report.

QoS Parameter Variance. Modal QoS shares range from 29% to 89% across the twelve scenarios, making sure that every fold carries genuine variation in declared reliability, durability, and priority. As noted in Section 4.1.1, one schema dimension (`max_blocking_ms_log`) remains zero throughout the corpus as a reserved extension point, while the declared deadline populates the other two (`has_deadline`, `deadline_ns_log`) on 75% of topics. Reported gains therefore stem from six active dimensions.

7.3.2. Convergent Validity Over Simulation Oracles

The three reliability-facing oracles measure distinct constructs, so we checked whether they agree before treating any as ground truth. Over the twelve inductive folds on the Application population, the behavioural queue-flow oracle and the topological cascade injector agree at mean Spearman $\rho = 0.627$ (top- K Jaccard 0.370 against 0.111 expected by chance), against I^* ’s own seed-to-seed test-retest of 0.811–1.000. The agreement is therefore substantial but distinctly below label noise, which is the reading we want: an oracle reproducing another to within its own reproducibility would be re-measuring the same topology rather than corroborating it. Two boundaries qualify this — a large share of the agreement is the two oracles concurring on which components are *harmless*, and I_{dyn} has a measured noise floor of its own that the headline does not correct for.

The Four Golden Signals captured during execution show the mechanism behind the separation: crashing a critical publisher degrades delivery to surviving consumers ($I_{\text{dyn}} > 0$) while *reducing* their queue waits through contention relief ($\rho = -0.499$ against tail-latency delta). Since I^* is a deterministic breadth-first reachability computation and I_{dyn} a stochastic queue simulation, their moderate agreement is convergent validity between two independent formulations rather than re-measurement, and it places the learned predictors as surrogates for topological cascade reach, not detectors of queue dynamics (Section 8.3).

7.3.3. Node-Type Stratification and Attention

One result governs how every other number in this paper is read. Measured against $I_{\text{comp}}(v)$ over the twelve scenarios of the corpus, stratified RM rank correlations are $\rho = 0.597$ (Application), 0.317 (Broker), and 0.138 (Node), while pooling all types gives $\rho = 0.217$ — less than two fifths of the Application figure it is supposed to summarise. This is why every evaluation is reported on a single stratum, and why pooled figures should be read as inflated wherever they appear. We claim only the weaker form: pooled correlation sits above the Execution Host stratum (0.138), so this is aggregation bias rather than a strict Simpson reversal, which holds on a companion study’s eight-scenario suite but not on this corpus. A global sensitivity sweep of

I_{comp} 's four severity weights ($N = 1,000$ Dirichlet draws; Supplementary Section S1.2) shows Application correlation exceeding pooled on *all* draws ($\rho \in [0.441, 0.599]$ vs. $[0.096, 0.351]$), while the strict condition holds on only 13.9% of the simplex. The stratification argument rests on the former, which is weight-invariant. A rule-based anti-pattern catalog on the same benchmark flags 93.4% of scored components and so does not discriminate; critical-set identification is delegated to the continuous rankers of Sections 7.1–7.2.

Aggregated by relation type over the ATM case study, first-layer mean HGT attention orders USES into libraries (0.227) above publish–subscribe channels (0.163–0.176). However, the spread across all seven relation types is narrow (0.15–0.23) and driven substantially by destination in-degree artifacts. Typed attention remains active across relation types without establishing a statistically distinct ordering.

7.4. RQ4: Out-of-Family Topological Transfer on Transcribed Open-Source Architectures

RQ4 Summary (Real-World Transfer):. Full-population transfer is established and substantial: under the protocol used everywhere else in this paper the learned model reaches $\rho = 0.760$ $[0.714, 0.819]$ against 0.51–0.53 for the three training-free baselines, with non-overlapping bootstrap intervals over the five systems. Active-stratum transfer is not established. Restricted to components that actually propagate failures, every predictor loses most of its correlation; the learned model is the only one whose point estimate stays positive ($\rho_{>0} = +0.236$ against -0.055 to -0.092), but its interval $[-0.053, +0.525]$ spans zero and so does every baseline's. Five systems cannot resolve this comparison, and we report it as unresolved rather than as a qualified positive. The one consistent pattern is directional: active correlation is positive on all three asynchronous pub-sub systems and non-positive on both synchronous call trees.

We evaluated the framework on five open-source distributed systems transcribed from public repositories: Online Boutique, Train-Ticket, Home Assistant, Autoware Universe (ROS 2), and EdgeX Foundry. All carry labels from the same simulation oracles used throughout, testing out-of-family topological transfer against simulated reachability rather than agreement with field incident telemetry.

Two evaluations are conducted. The first is an exploratory evaluation of the closed-form explanation layer $Q(v)$ against the composite oracle $I_{\text{comp}}(v)$ ($\rho = 0.514$ – 0.800); as noted in Section 4.3, I_{comp} 's four severity weights are unswept heuristics. The second, Section 7.4.1, evaluates learned predictors zero-shot against $I^*(v)$ (where RM achieves a lower mean $\rho = 0.516$). Topo-QoS is scored here on the flow projection, for the reason given in Section 8.4. Standard middleware default contract profiles (e.g., ROS 2 Best-Effort/Reliable, MQTT QoS 0/1) were applied uniformly across edges lacking explicit manifests to ensure identical graph representations across baselines.

7.4.1. Zero-Shot Transfer of the Learned Model

To assess generalization outside the generator, HGT-QoS was trained on all twelve synthetic scenarios and evaluated zero-shot across five open-source systems versus $I^*(v)$ (five seeds; no open-source graph contributed training gradients or checkpoint selection). Tables 9–10 report this arm at the 3-layer, 300-epoch budget used for every other learned result in this paper. An earlier configuration used 2 layers and 150 epochs, chosen to limit over-smoothing across the smaller diameters of these transcribed meshes ($|V_{\text{app}}| \leq 41$); that reasoning appeals to a property of the evaluation targets, so although no target label or gradient reached the model, the configuration was not blind to the test systems and we do not report it as the primary result. It is uniformly slightly stronger ($\rho = 0.792$ against 0.760; $\rho_{>0} = +0.281$ against $+0.236$; $F_1 @ K = 0.533$ against 0.470) and leaves every qualitative conclusion unchanged, which is the sensitivity we draw from it.

Key Insights for Real-World Transfer:

1. **The full-population figure is not a ranking result.** On all Applications, HGT-QoS reaches $\rho = 0.760$ $[0.714, 0.819]$ against 0.511 (Topo), 0.526 (Topo-QoS) and 0.516 (RM), leading on 4 of 5 systems with a non-overlapping interval. However, between 29% (Home Assistant) and 66% (Train-Ticket) of Applications carry zero simulated impact, so full correlation rewards separating inert components from active ones rather than ranking the active ones.
2. **On the active stratum, five systems cannot resolve the question.** The learned model is the only predictor whose active-stratum estimate stays positive ($\rho_{>0} = +0.236$ against -0.055 to -0.092), but its interval $[-0.053, +0.525]$ spans zero and so does every baseline's (Table 10). We therefore claim

Table 9: Zero-shot out-of-family transfer to five transcribed open-source systems, scored against $I^*(v)$ on the Application population under the protocol used throughout this paper (3 layers, 300 epochs; five seeds, $\pm$ = spread over seeds). No open-source graph contributed gradients or checkpoint selection. Training-free references are deterministic and are scored on identical labels, populations and node sets. Systems are grouped by communication paradigm: active-stratum correlation $\rho_{>0}$ is positive on all three asynchronous pub-sub systems and non-positive on both synchronous call trees. Where a system declares no explicit QoS manifest, default middleware contract profiles (ROS 2 Best-Effort/Reliable, MQTT QoS 0/1) are applied uniformly across baselines. Identification metrics for the same runs are in Supplementary Table S13.

Real-World Architecture	$ V_{\text{app}} $	$n_{>0}$	RM (ρ)	Topo (ρ)	Topo-QoS (ρ)	HGT-QoS (ρ)	HGT-QoS ($\rho_{>0}$)
<i>Asynchronous pub-sub systems — forward failure cascades</i>							
Autoware.universe (ROS 2)	32	19	0.357	0.307	0.378	0.716 $\pm$ 0.081	+0.517
EdgeX Foundry (Industrial IoT)	22	10	0.470	0.534	0.534	0.793 $\pm$ 0.037	+0.183
Home Assistant (Smart Home)	24	17	0.265	0.297	0.289	0.864 $\pm$ 0.063	+0.702
<i>Synchronous microservice call trees — backward timeout cascades</i>							
Online Boutique (microservices)	22	8	0.777	0.891	0.888	0.710 $\pm$ 0.070	−0.031
Train-Ticket Booking Mesh	41	14	0.713	0.528	0.541	0.717 $\pm$ 0.096	−0.192
Mean	—	—	0.516	0.511	0.526	0.760	+0.236

Table 10: Means over the five transcribed systems on the full Application population (ρ) and restricted to components with strictly positive ground-truth impact ($\rho_{>0}$), with percentile bootstrap intervals over the five systems ($B = 2,000$). All four predictors are scored on identical labels, populations and node sets from one run, so the columns are commensurable. At $n = 5$ the intervals are descriptive and carry no significance claim. **The full-population intervals separate the learned model from all three baselines; every active-stratum interval spans zero**, which is why RQ4 is reported as established on the full population and unresolved on the active one.

Predictor	ρ (full), 95% CI	$\rho_{>0}$ (active), 95% CI
RM / $Q(v)$	0.516 [0.343, 0.680]	−0.055 [−0.292, +0.213]
Topo	0.511 [0.346, 0.703]	−0.083 [−0.269, +0.108]
Topo-QoS	0.526 [0.357, 0.699]	−0.092 [−0.268, +0.094]
HGT-QoS	0.760 [0.714, 0.819]	+0.236 [−0.053, +0.525]

only that its point estimate is positive, not that it transfers. The one consistent pattern is directional: $\rho_{>0}$ is positive on all three asynchronous pub-sub systems (Home Assistant +0.702, Autoware +0.517, EdgeX +0.183) and non-positive on both synchronous call trees (Online Boutique −0.031, Train-Ticket −0.192). A 3–2 split is a pattern worth naming, not a tested effect.

- Identification is where the learned model separates most clearly.** On top- K overlap it averages 0.470 [0.410, 0.540] against 0.248 [0.09, 0.46] for the structural baselines, whose interval overlaps it; on the threshold-free measures of Supplementary Table S13 the gap is wider ($F_1@7$ 0.473 vs. 0.29–0.33; PR-AUC 0.713 vs. 0.47–0.52). The mechanism is visible on EdgeX, where symmetric star connections from peripheral adapters to brokers produce identical betweenness ties that collapse structural triage entirely ($F_1@K = 0.000$) while relational attention still separates components.
- Architectural boundary condition and ingestion boundary.** Synchronous RPC architectures propagate failures *backward* along invocation trees via timeouts and thread starvation [48], whereas pub-sub architectures cascade forward via queue saturation. Trained on pub-sub semantics, the model’s directional bias does not transfer to call trees. We therefore state an ingestion boundary: SaG’s learned pipeline applies to asynchronous, event-driven architectures (ROS 2, Kafka, DDS, MQTT), and static call-graph reachability tools should be used for synchronous RPC/REST meshes. The closed-form baselines are not the substitute there either — **Topo-QoS** leads on Online Boutique only on the full population (0.888) and is negative on its active components (−0.072), so its advantage lies in identifying inert services rather than ranking propagating ones.

7.5. RQ5: Analysis Cost and Its Comparison Against Simulation

RQ5 Summary (Analysis vs. Simulation Cost): While the neural forward pass executes in milliseconds (56 ms for 2,000 nodes), cold deterministic feature extraction requires up to 239.3 s, dominated by the $O(|V|^2 + |V||E|)$ Connectivity Degradation Index. Cold static gating costs 2–18 $\times$ the discrete-event simulation it was meant to displace across the twelve scenarios (median 5.6 $\times$), reaching its maximum only on the 520-node Enterprise

mesh (79.3 s vs. 4.5 s). The premium tracks the density of the derived dependency projection rather than component count, and at no scenario size is the gate cheaper — refuting the assumption that static analysis is inherently cheaper than simulation without incremental graph caching.

RQ5 quantifies computing overhead and sustainability during CI/CD evaluation, with per-stage latencies reported in Table 11:

Table 11: Per-stage latency of the inference pipeline across scaling graph sizes (CPU, median of 3 runs; 5 for the forward pass). The analysis stage is stable across repeats (p10–p90 within 1% of the median everywhere) while the forward pass is not, which is why its column carries a spread: at 56 ms the measurement is dominated by interpreter and dispatch overhead rather than by the graph.

$ V $	$ E $	Analyze (s)	Graph → Tensor (s)	HGT Forward (ms)	Analyze : Forward	Forward p10–p90 (ms)
249	1,127	1.74	0.010	26.5	66×	13.0–34.4
499	2,402	8.32	0.022	16.4	509×	15.6–16.4
999	6,422	44.54	0.056	21.1	2,108×	19.1–36.5
1,998	19,301	239.34	0.157	56.2	4,259×	43.8–57.8

The neural stage is the cheapest by a wide margin and the deterministic one is not. At 2,000 components the HGT forward pass takes 56 ms against 239 s for structural analysis, a ratio of 4,259× — but that is a ratio between pipeline stages, not a cost of evaluation, because indices 0–17 of every node feature vector are produced by the analysis stage the forward pass depends on. End-to-end evaluation of an unseen 2,000-component architecture takes about four minutes, of which the learned model is 0.02%; the 56 ms is the marginal cost of re-scoring an already-analysed graph. Across the corpus the complete gate (structural analysis plus 18 anti-pattern detectors, whose share is ≤ 0.19 s) executes in 0.16–79.3 s (Table 12).

One metric dominates cost. Across the endpoints of Table 11 the measured cost is consistent with the stage’s $O(|V|^2 + |V||E|)$ bound: from 249 to 1,998 components wall-clock rises 138× against a 137× growth in $|V||E|$. That is an endpoint coincidence rather than a tracked curve: between the middle rows the component count doubles (499 → 999) while wall-clock rises 5.4×, faster than the bound requires, and the series varies $|V|$ and $|E|$ together so it cannot separate them — Table 12 does that on the corpus. The dominant term is the Connectivity Degradation Index, computed for every node in the main connected component rather than for articulation points alone. That is a correctness requirement, not an oversight: gating CDI to articulation points leaves it identically zero wherever removal does not literally disconnect the graph, driving $A(v)$ to a near-constant in exactly the redundant multi-publisher topologies this system targets. The cost is the price of a non-degenerate Availability score.

7.5.1. The Gate Is Not Cheaper Than the Simulation It Replaces

The framing that motivated this analysis — static gating as a low-cost substitute for dynamic simulation — does not survive measurement against our own oracle. Timing the cascade reachability labeling sweep (five seeds, node types Application/Broker/Library, the full ground-truth run) over all twelve scenarios, on the same machine, at the same commit and in the same measurement session as the gate, gives 0.08–4.49 s per scenario against 0.16–79.3 s for the analysis gate. Both maxima occur in the 520-component Enterprise mesh, so the largest scenario compares 4.5 s of simulation against 79.3 s of static analysis: there the gate costs roughly eighteen times as much as the simulation it is meant to replace. That is the extreme, not the centre. Pairing the sweeps scenario by scenario (Table 12) gives 2.0–17.7×, median 5.6×. Two things follow, in opposite directions: the headline is weaker than one number suggests, and the finding is stronger, because the gate is more expensive on *all twelve* scenarios. What predicts the premium is the derived projection’s size, not the component count — the ratio correlates with $|E_{\text{proj}}|$ at $\rho = 0.95$ against 0.79 for cost against $|V|$. Enterprise is the outlier because its 300 applications share only 120 topics, so Rule 1 derives a near-complete graph of 26,276 edges, while IoT Smart City has more components, an eighth of the edges and a third of the cost. These are wall-clock figures on one commodity CPU, read to one significant figure: across sessions the gate maximum ranges over 77–83 s and the oracle maximum over 4.5–4.8 s, giving 16.7× and 17.7× on two independently paired sessions. Two earlier pairings are superseded rather than reconciled, one unstamped and one pairing this corpus’s oracle against a companion study’s eight-scenario gate; Table 12 uses only the twelve-scenario pairing, whose halves share a commit and corpus digest.

Table 12: Static analysis gate against the cascade-reachability oracle, per scenario, from one paired measurement session. **Gate** is structural analysis plus the 18 anti-pattern detectors, whose own share is negligible (≤ 0.19 s everywhere). **Oracle** is the full five-seed ground-truth labelling sweep over Application, Broker and Library nodes. $|E_{\text{proj}}|$ is the number of derived **DEPENDS_ON** edges in the Application–Library projection the analysis stage traverses, read from the committed cache. The ratio is 2.0–17.7 $\times$ with a median of 5.6 $\times$: the eighteen-fold figure quoted elsewhere in this section is the maximum, not the typical case. Ordering by $|E_{\text{proj}}|$ rather than by $|V|$ is what makes the column monotone — Enterprise carries 300 applications over only 120 topics, so its Rule-1 projection is near-complete, while IoT Smart City has more components and a sixth of the edges at a third of the cost.

Scenario	$ E_{\text{proj}} $	Gate (s)	Oracle (s)	Ratio
Enterprise	26,276	79.27	4.487	17.7 $\times$
Enterprise Integration (ESB)	4,641	3.17	0.245	12.9 $\times$
AV System	3,073	2.61	0.355	7.4 $\times$
Financial Trading	2,657	1.71	0.239	7.1 $\times$
Real-Time Gaming	2,318	2.43	0.379	6.4 $\times$
Healthcare	1,590	0.92	0.149	6.2 $\times$
IoT Smart City	1,835	5.48	1.104	5.0 $\times$
Telecom RAN	2,116	3.70	1.065	3.5 $\times$
Logistics Fleet	1,660	2.74	0.939	2.9 $\times$
Microservices (synthetic)	1,524	2.29	0.828	2.8 $\times$
Industrial SCADA	897	2.43	1.191	2.0 $\times$
ATM System	155	0.16	0.080	2.0 $\times$
Median	—	—	—	5.6 $\times$

This finding refutes the assumption that static analysis is computationally cheaper than in-process simulation: breadth-first cascade traversal is simpler than computing all-pairs connectivity degradation ($O(|V|^2 + |V||E|)$). Two deployment trade-offs survive it. Static analysis scores infrastructure components (physical execution hosts, network interconnects) and dependency edges that node-level message flow simulation leaves unscored without dedicated physical failure models, and its deterministic metrics can in principle be cached across commits, recomputing only the k -hop neighbourhood a pull request touches. Table 11 times full from-scratch recomputation; caching is an engineering hypothesis not yet implemented, and without it direct simulation is strictly faster.

8. Discussion, Threats to Validity, and Limitations

8.1. Discussion and Practical Consequences

When to use Topo-QoS, and when to use HGT-QoS. The findings do not support an unequivocal recommendation of the learned model over the closed-form alternative; we present empirical trade-offs directly.

- **Training-free ranking (Topo-QoS) vs. Simulation.** Topo-QoS requires no training or checkpoints, reaching $\rho = 0.553$ zero-shot across twelve synthetic architectures and 0.526 across five open-source ones, with no learned advantage ($+0.085$, $p = 0.151$). This equality reflects domain inductive bias: in asynchronous pub-sub architectures lacking severe backpressure, cascading failure is reachability-driven (recovered at $\rho = 0.965$ by topology alone), leaving relational GNNs minimal headroom over path centrality. On large diameters ($N \geq 500$, Enterprise), global paths resolve bottlenecks that 3-hop convolutions localize (0.795 vs. 0.461). It is thus the zero-maintenance default for CI; where queue parameters exist ($N < 150$), direct simulation (0.08–4.5 s) provides exact ground truth.
- **When to deploy HGT-QoS.** HGT-QoS is justified when systems feature dense, irregular topologies with multi-tenant shared library or host dependencies under $N < 150$, where relational attention gains $+0.210$ to $+0.229$ over Topo-QoS (the synthetic Microservices and ATM folds, Section 7.2.1; note that on the transcribed Online Boutique mesh the ordering reverses on the full population, 0.769 against 0.888), and when teams require scoring of unsimulated non-runnable infrastructure entities (physical execution hosts, hardware network links) that node-level message flow simulation leaves unscored without dedicated physical failure models. Otherwise, untyped QoS-weighted GNNs offer a better

efficiency trade-off ($\rho = 0.604$ with 28,168 parameters vs. HGT-QoS’s 0.638 with 434,620), as combining typing and QoS yields minimal additive benefit.

- **Capabilities without a closed-form counterpart.** Per-relation attention (Section 7.3.3) offers a typed diagnostic, but measured spread across relation types is narrow (0.15–0.23) and governed by destination in-degree, making it active but diagnostically unreliable on present evidence.
- **Ingestion boundary and active-stratum reality.** Deployment must condition on communication synchrony (Table 13): on asynchronous pub-sub systems (ROS 2, DDS, MQTT, Kafka), HGT-QoS transfers positively on active components ($\rho_{>0} \in [+0.183, +0.702]$), whereas on synchronous call trees it inverts (−0.031 to −0.192) and should not be deployed — use static call-graph tools instead. While full-population transfer isolates inert services ($\rho = 0.760$), zero-shot ranking of active components remains unresolved without runtime telemetry.

Architectural and Systemic Drivers of Graph Learning Success. Across seventeen architectures, graph neural network performance is governed by the five systemic factors in Table 13.

Table 13: Systemic drivers of graph-learning performance among the seventeen evaluated architectures, with the mechanism each reflects and the evidence for it on this corpus.

Factor	Mechanism	Evidence on this corpus
Communication synchrony	Directional bias must match physical failure flow: pub-sub starves forward, synchronous call trees time out backward [48].	Pub-sub $\rho_{>0} > 0$ on 3/3; RPC non-positive on 2/2, with every training-free baseline negative on both.
Scale and diameter	Fixed 3-layer message passing covers the diameter of small graphs ($N < 150$) but localizes at $N \geq 500$.	520-node Enterprise: 0.461 vs. 0.795. This is a single-scale scenario, so the threshold is a conjecture, not a measured breakpoint.
Topology and symmetry	Isomorphic 1-hop neighborhoods yield near-identical embeddings; dense irregular meshes do not.	Broker-hub ESB $\rho = 0.472$; on the synthetic Microservices and ATM folds, +0.210 to +0.229 over Topo-QoS. Single-fold observations.
Relational heterogeneity	Typing is essential where shared hosts or libraries induce blast radii, but substitutes for QoS once contracts are declared.	Untyped GNNs drop to $\rho = 0.317$ (Section 7.2).
Inert-node base rates	Zero-impact sinks inflate full-population correlation through trivial inertness filtering.	21%–52% of applications carry $I^*(v) = 0$; $\rho_{>0}/\rho \approx 51\%$ –56%.

Dual-Engine Consensus Protocol. In pre-deployment continuous integration, an automated heuristic fallback that conditionally routes predictions to a closed-form baseline based on label-free prediction dispersion $\hat{\sigma}$ is unviable because prediction variance exhibits a slightly negative correlation with out-of-distribution advantage ($\rho_s = -0.126$, Section 7.2.1). Rather than relying on fragile single-model uncertainty heuristics, SaG tackles this boundary through a principled **Dual-Engine Consensus Protocol**. The pipeline evaluates the learned relational model (HGT-QoS) and the closed-form topological baseline (Topo-QoS) concurrently. Entities identified in the top- K critical set by both engines are classified as unanimous high-priority risks, while rank divergences are surfaced for architectural review. Given non-zero retraining drift across GPU sweeps (0.041 on fold mean, Section 8.3), production CI/CD deployments should freeze version-controlled model checkpoints rather than retraining on commit hooks.

Role of the Explanation Layer. The RM profile ($Q(v)$, Section 5) decomposes ISO/IEC 25010 structural mechanics to direct targeted refactoring (distinguishing Availability replication from Fault-Tolerance circuit breakers), while neural and centrality predictors govern triage priority. Its elicited AHP weights trail a uniform prior, which we recommend pending user studies.

Practitioner Decision Matrix. Table 14 provides an operational decision guide summarizing when each analytical instrument in the SaG portfolio should be applied.

8.2. Performance and Computational Sustainability Implications

What sustainability means for a pre-deployment gate. Green software engineering assesses energy across development, assurance, and execution [29, 88, 89, 90, 91, 30, 31]. Live chaos sweeps require cluster-hours across provisioned nodes; pre-deployment static analysis avoids provisioning clusters entirely. We

Table 14: Operational decision matrix for applying the Software-as-a-Graph method portfolio across software engineering workflows.

Instrument	Target Architecture / Context	Operational Role and Output
HGT-QoS (Learned Relational)	Asynchronous pub-sub meshes under distribution shift	Forecasts multi-hop cascade blast radii across topics, brokers, and shared libraries ($\Delta\rho = +0.234$).
Topo-QoS (Closed-Form)	Lightweight CI/CD commits, resource-limited environments	Instantaneous, training-free ranking ($\rho = 0.553$) without GPU requirements or checkpoint drift.
Dual-Engine Consensus	Production gating and deployment-readiness sign-offs	Concurrently executes HGT-QoS and Topo-QoS ; prioritizes unanimous top- K components for triage.
RM Explanation Layer	Architectural refactoring and root-cause analysis	Standardized ISO/IEC 25010/25019 structural attribution (Availability vs. Fault-Tolerance vs. Maintainability).

state the benefit as infrastructure avoidance rather than measured energy savings, since we did not profile avoided execution. We bound the expended computation: taking Table 12’s wall-clock at base SoC power (28 W), one pass over all twelve scenarios costs at most 3.0 kJ (0.83 Wh), with Enterprise accounting for 2.2 kJ, against ≤ 0.31 kJ (0.086 Wh) for the oracle sweep. These upper bounds overstate draw because single-threaded execution shares package power and includes interpreter I/O. Direct joule profiling via RAPL/NVML [30, 32, 33] remains future work; gate energy is negligible except for staging clusters (< 1 Wh), but it is not an empirical reduction.

Where the cost actually sits. We withdraw the efficiency claim: cold static analysis does not reduce CPU computation over simulation. The gate costs $2\text{--}18\times$ the cascade traversal across the twelve scenarios (median $5.6\times$; Table 12), rising to its maximum on Enterprise (79.3 s against 4.5 s) where CDI’s $O(|V|^2 + |V||E|)$ cost meets the densest derived projection in the corpus. It is more expensive in every scenario, so the comparison direction does not depend on which one is quoted. In CI, sustainability requires incremental caching: caching base metrics across commits and extracting features only for PR-delta subgraphs decreases latency to a sub-second forward pass (56 ms). We emphasize this is an engineering hypothesis: delta latency depends on the PR diff’s topological centrality (leaf worker vs. central broker), and we focus on benchmarking commit-diff extraction across PR topologies.

8.3. Threats to Validity

Construct Validity. Ground-truth impact $I^*(v)$ derives from simulation rather than live outages. While I^* correlates with dynamic queue flow I_{dyn} ($\rho = 0.627$ against a 0.811–1.000 ceiling, Section 7.3.2), top- K Jaccard reaches only 0.27–0.37, and $I^*(v)$ is recovered at $\rho = 0.965$ by topology-only relabeling, confirming the target captures reachability rather than runtime queue collapse or retry storms. More fundamentally, $I^*(v)$ is a deterministic functional of the graph the predictors read, so RQ1 asks a learned model to recover a closed-form graph computation from features that already summarise that graph — a task on which a strong closed-form comparator is expected, which bounds what the parity result says about graph learning generally. The remedy is internal: I_{dyn} agrees with I^* at only $\rho = 0.627$ and so carries the non-topological structure this design lacks. Retargeting the LOSO contrasts on it needs per-node predictions at the twelve-fold artifact’s fidelity, so it requires a full re-run and is the first experiment we intend to report. Validating topological cascades against production incident telemetry remains the central empirical frontier. Transcribing the open-source architectures also relied on standardized heuristics applied by a single coder, so there is no inter-coder agreement figure. Those five systems carry the whole of RQ4, and a transcription choice that systematically favored one predictor would be invisible to us; independent re-transcription of at least two of them is required before these numbers read as firmer than indicative.

Internal Validity. Feature leakage is prevented by strict graph separation: predictors consume G_{analysis} , while simulation oracles traverse $G_{\text{structural}}$ (CI-asserted). Parity is maintained via matched datasets, depths, and early stopping. Comparing **HGT-QoS** to **GAT-N-QoS** conflates typing with a $15.4\times$ capacity gap (434,620 vs. 28,168) and bidirectional passing (103,725 reverse parameters in **HGTConv**), which four registered control arms would isolate (**GAT-N-C**, **GAT-N-QoS-C**, **GAT-N-QoS16-C**, **HGT-QoS-U**; Section 8.4). They are implemented in the replication package and registered with a decision rule fixed in advance, but none has been run at the

reported budget, so this confound is disclosed rather than resolved. Six active QoS dimensions govern profiles, with one reserved extension point.

External Validity. The corpus comprises 2,812 components across seventeen architectures spanning six domains: robotics (ROS 2), smart cities, healthcare, finance, enterprise ESB, and edge IoT (EdgeX Foundry, Home Assistant). Heterogeneity exposes the active-stratum boundary condition: pub-sub cascades propagate forward via starvation ($\rho_{>0} \in [+0.183, +0.702]$, Table 9), whereas synchronous RPC microservices cascade backward along call trees via timeouts [48], where active correlation inverts (-0.031 to -0.192) and baselines fail (Table 10). Directional biases must condition on synchrony. Scaling beyond 2,000 nodes requires incremental caching or mini-batching (GraphSAINT [92]).

Conclusion Validity. Distributions are evaluated using a non-parametric correlation (Spearman ρ), bootstrap intervals ($B = 2,000$), and Wilcoxon signed-rank tests. Analyses are stratified because pooling node types conflates disparate strata (Execution Host $\rho = 0.138$ vs. Application $\rho = 0.597$), leaving the pooled figure (0.217) at less than two-fifths of the evaluated stratum. Because folds share training graphs, nominal Wilcoxon p -values are optimistic and interpreted descriptively alongside 12-fold sign consistency; synthetic graphs derive from a single generator family.

Repeatability and GPU Re-Baseline. Seed spread and run-to-run displacement are distinct quantities; only the former is reported in Table 6. At fixed code and seeds, the pipeline is deterministic: consecutive runs reproduced every reported figure identically. Across hardware platforms and code revisions, displacement is non-zero. Between the two most recent committed sweeps — transitioning from initial CPU runs to a full clean GPU (CUDA) re-baseline on an identical corpus digest — all 180 training-free cells reproduced identically ($|\Delta\rho| = 0.000000$), confirming absolute determinism of graph extraction. In contrast, the 240 learned cells shifted by up to 0.519 at seed level and 0.172 at fold mean (GAT-N on Healthcare, $0.600 \rightarrow 0.428$). This displacement stems from three documented technical factors: (1) resolving an upstream PyTorch Geometric device-placement bug in HGTConv (`saag/prediction/models/core.py`) that was masked on CPU; (2) eliminating stale checkpoint resumption artifacts by enforcing clean cache regeneration across all 12 folds; and (3) hardware non-determinism in CUDA atomic scatter/gather reductions during message passing. Crucially, under clean full GPU convergence, HGT-QoS regularly outperforms HGT (0.638 vs. 0.551), while displacement is smallest for HGT-QoS (0.041 on fold mean). Read the sign of that result as robust and its magnitude as pinned to these artifacts. The Table 8 contrasts are better placed, being differences within one sweep. The per-arm ledger is in the replication package (`reproduce/rerun_drift.py`). Revisions also altered the active stratum ($n_{>0}$ shift 26%–56%), making $\rho_{>0}$ non-comparable across revisions. Learned figures are pinned to commit artifacts; provenance tracking in Data Availability makes this auditable.

8.4. Limitations and Future Work

Correction of the Real-World Baseline. On raw multigraphs, betweenness vanishes for Application nodes; evaluating on DEPENDS_ON flow projections enables Topo-QoS across all five real-world systems (Table 9).

Explanation Layer Validation. The explanation layer is presented as a design pattern rather than a validated contribution (Section 5): its elicited AHP weights rank worse than a uniform prior (Section 7.3), it trails unweighted centrality, and no user study has tested whether its archetypes help a developer act. We recommend the uniform prior and retain the elicited constant as the shipped default for one reason, stated. Hence, this is not inertia: the same shrinkage constant parameterizes the I_{comp} severity weights, so moving it would silently relabel the composite oracle and every figure scored against it. Decoupling the two, then a mutation benchmark (inject a known single point of failure or cascade hub and test whether the layer names the right archetype) and a developer study, are the prerequisites for any evidential claim here. **Uncontrolled Confounds in Typing.** Table 8 holds substrate, depth, and training splits constant, but capacity (434,620 vs. 28,168) and reverse directionality (103,725 parameters in HGTConv) remain unmatched; the +0.134 effect reflects the joint transition to HGT. Four control arms that isolate these factors — GAT-N-C and GAT-N-QoS-C (capacity-matched untyped models), GAT-N-QoS16-C (edge-channel width) and HGT-QoS-U (unidirectional passing) — are implemented in the replication package and registered in the analysis plan together with a decision rule fixed before any of them was run. None has yet been run at the reported budget. Running them is the single change that would most alter what Section 7.2 is entitled to claim, and the registered rule commits us to narrowing that claim if the margin does not survive. **No Comparison Against a Published**

Method. Every comparator in this paper is either our own closed-form score or an ablation of our own model. Section 2.4 reviews learned node-criticality models (FINDER [64], DrBC [65], PowerGraph [66]) without replicating any of them, so the RQ1 finding is a statement about this heterogeneous transformer against these baselines, not about graph learning for criticality ranking in general. Reproducing DrBC or FINDER on this corpus — both address the ranking task directly — is the comparison that would generalize the claim, and its absence is the clearest gap in the evaluation. **Absence of Live Incident Telemetry.** Simulation oracles (I^* , I_{comp} , I_{dyn} , I_M) ensure determinism but do not replace incident logs or chaos tests; validating topological cascades against production outages is necessary. **Model Selection and Incremental Caching.** Early stopping uses an inner validation split; held-out scenario validation is a prioritized extension. Prediction dispersion fails as an out-of-distribution fallback signal (Section 7.2.1). Practical CI deployment requires incremental graph caching over PR diffs. **Future Directions.** Key extensions include: (1) modeling hybrid distributed architectures (e.g., synchronous REST/gRPC frontends feeding asynchronous Kafka pipelines) by conditioning edge directional semantics on synchrony (SYNCHRONOUS_CALL with backward timeout cascading vs. PUBLISH_SUBSCRIBE with forward starvation); (2) modeling distributed LLM serving backbones (vLLM, DeepSpeed); (3) measuring hardware energy via RAPL/NVML in joules, replacing the bounded estimate of Section 8.2; (4) retargeting the RQ1 and RQ2 contrasts on I_{dyn} , whose queueing structure is not recoverable in closed form from the input graph, so that the comparison tests graph learning rather than the recovery of a topological functional; (5) separating whether $w(e)$ carries contract semantics or acts as a traffic proxy, by holding the declared QoS triple fixed while varying payload size and publication frequency and scoring Topo-QoS against a topology-only relabelling of the oracle; (6) reproducing a published learned-criticality baseline; and (7) advancing to prescriptive synthesis generating pull requests with circuit breakers.

9. Conclusion

This study explored how well heterogeneous graph neural networks can support pre-deployment dependability analysis in asynchronous distributed systems. We used Software-as-a-Graph (SaG) to turn Architecture-as-Code manifests into typed multigraphs. We then compared a relation-specific Heterogeneous Graph Transformer with QoS edge encodings to unparameterized structural baselines and an ISO/IEC 25010/25019 attribution layer without runtime telemetry. Our main question was whether relation typing and QoS encodings improve prediction when used together or separately.

The main finding is that relation typing and 16-dimensional QoS edge encodings act as substitutes rather than working together. Each one has a significant effect under inductive distribution shift ($\Delta\rho = +0.134$ and $+0.187$, Holm $p = 0.0015$, with typing indicating the move to HGT), but when combined, their interaction is negative (-0.199 across all twelve folds, $p = 0.0005$; -0.233 after a Fisher transformation). In other words, the sub-additivity comes from the mechanisms themselves, not the metric: typing adds $+0.234$ without QoS and only $+0.035$ with it. Using both together gives little extra benefit, which suggests both channels encode similar traversal semantics for a nearly topological simulation target (recovered at $\rho = 0.965$ without QoS). Typing also brings a $15.4\times$ capacity gap and changes message-passing directionality.

These outcomes have practical consequences for distributed software engineering: (1) In complex asynchronous publish-subscribe event meshes like ROS 2, DDS, or MQTT, HGT-QoS gives important relational inductive bias under distribution shift, achieving strong zero-shot transfer on active failure spreaders ($+0.517$ on Autoware, $+0.702$ on Home Assistant) where training-free baselines fail; (2) for lightweight CI/CD gating with limited resources, the closed-form Topo-QoS baseline ($\rho = 0.553$) offers instant deterministic ranking with no training or GPU use; (3) for production pre-deployment gating, the Dual-Engine Consensus Protocol combines both tools, prioritizing components that both rank as top- K critical and highlighting ranking differences for expert review; (4) the ISO/IEC 25010 explanation layer provides clear root-cause attribution to guide targeted refactoring (Availability replication vs. Fault-Tolerance circuit breakers) instead of continuous ranking. Meanwhile, while forward neural inference runs in 56 ms, deterministic feature extraction takes 2 to 18 times longer than the simulation it replaces (median $5.6\times$ over twelve scenarios). This shows that static analysis helps avoid extra infrastructure (like live staging clusters) rather than providing a raw CPU speed advantage, unless used with incremental topology caching. Two main following steps remain: internally, the predictors are currently scored against a deterministic function of their own input, so the next test is to

compare them to the queue-flow oracle, whose behavior cannot be recovered from the graph in closed form. Externally, and more importantly, none of these outcomes have been validated against real failures. Whether these pre-deployment pipelines can predict live production outages is still the main empirical question, and this study does not answer it.

Declarations

CRediT Authorship Contribution Statement. **Ibrahim Onuralp Yigit:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Feza Buzluca:** Conceptualization, Methodology, Validation, Writing – review and editing, Supervision. **Declaration of Competing Interest.** The authors declare no competing financial interests or personal relationships that could have influenced this work. **Funding.** This research received no external grant. **Data Availability.** The replication package (datasets, harnesses, checkpoints, scripts) is available on Zenodo under DOI 10.5281/zenodo.14922108 [93] with `uv/pip` environments. Synthetic datasets regenerate byte-identically. The deposit ships all artifacts backing reported tables as a dated bundle (`SaG_JSS_Results_<stamp>`) with a `MANIFEST.json` recording SHA-256 digests, commit hashes, and corpus provenance. Five artifacts carry no provenance block, where correspondence is asserted by the bundle. The verification script (`reproduce/reconcile_manuscript.py`) runs standalone against the deposit, mechanically verifying all 415 table quantities against JSON artifacts.

Declaration of Generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Anthropic’s Claude to assist with typesetting, LaTeX formatting, and the development of analysis and reporting scripts in the replication package. After using this tool the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. The study design, the choice of experiments, the interpretation of results, and all scientific claims are the authors’ own.

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074.
- [2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.
- [3] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).
- [4] OASIS, MQTT version 5.0, OASIS Standard, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (accessed 9 September 2026) (2019).
- [5] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O’Reilly Media, 2015.
- [7] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Computing Surveys* 35 (2) (2003) 114–131.
- [8] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, *Physical Review E* 66 (2002) 065102(R).
- [9] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, *Nature* 464 (2010) 1025–1028.
- [10] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, *Nature* 406 (2000) 378–382.
- [11] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, *IEEE Transactions on Dependable and Secure Computing* 1 (1) (2004) 11–33.
- [12] L. Bass, P. Clements, R. Kazman, *Software Architecture in Practice*, 3rd Edition, Addison-Wesley, 2012.
- [13] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).
- [14] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).
- [15] D. E. Perry, A. L. Wolf, Foundations for the study of software architecture, *ACM SIGSOFT Software Engineering Notes* 17 (4) (1992) 40–52.
- [16] W. Cunningham, The WyCash portfolio management system, in: *Addendum to the Proc. Conf. on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, 1992, pp. 29–30.

- [17] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR), 2009, pp. 255–258.
- [18] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS), 1998, pp. 68–78.
- [19] SonarSource, Clean as you code, SonarQube documentation, <https://docs.sonarsource.com/sonarqube-server/latest/core-concepts/clean-as-you-code/introduction/> (accessed 9 September 2026) (2024).
- [20] T. J. McCabe, A complexity measure, IEEE Transactions on Software Engineering SE-2 (4) (1976) 308–320.
- [21] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, IEEE Transactions on Software Engineering 20 (6) (1994) 476–493.
- [22] N. Fenton, J. Bieman, Software Metrics: A Rigorous and Practical Approach, 3rd Edition, CRC Press, 2014.
- [23] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, IEEE Software 33 (3) (2016) 35–41.
- [24] L. C. Freeman, A set of measures of centrality based on betweenness, Sociometry 40 (1) (1977) 35–41.
- [25] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, Computer Networks and ISDN Systems 30 (1–7) (1998) 107–117.
- [26] U. Brandes, A faster algorithm for betweenness centrality, Journal of Mathematical Sociology 25 (2) (2001) 163–177.
- [27] M. E. J. Newman, Networks: An Introduction, Oxford University Press, 2010.
- [28] I. O. Yigit, F. Buzluca, A graph-based dependency analysis method for identifying critical components in distributed publish-subscribe systems, in: Proc. IEEE Int. Conf. on Recent Advances in Systems Science and Engineering (RASSE), 2025, pp. 1–8. doi:10.1109/RASSE64831.2025.11315354.
- [29] C. Calero, M. Piattini (Eds.), Green in Software Engineering, Springer, Cham, Switzerland, 2015. doi:10.1007/978-3-319-08581-4.
- [30] L. Lannelongue, J. Grealey, M. Inouye, Green algorithms: Quantifying the carbon footprint of computation, Advanced Science 8 (12) (2021) 2100707. doi:10.1002/adv.202100707.
- [31] R. Verdecchia, J. Sallou, L. Cruz, A systematic review of Green AI, WIREs Data Mining and Knowledge Discovery 13 (4) (2023) e1507. doi:10.1002/widm.1507.
- [32] V. Schmidt, K. Goyal, A. Joshi, B. Feld, L. Conell, N. Laskaris, D. Sarthou, H. Verreault, J. Blank, S. Zhang, Codecarbon: Estimate and track carbon emissions from machine learning computing, Journal of Open Source Software (2021).
- [33] A. Noureddine, Joularjx: A java-based software power monitoring tool, SoftwareX 14 (2021) 100688.
- [34] R. C. Cheung, A user-oriented software reliability model, IEEE Transactions on Software Engineering SE-6 (2) (1980) 118–125.
- [35] K. Goseva-Popstojanova, K. S. Trivedi, Architecture-based approach to reliability assessment of software systems, Performance Evaluation 45 (2–3) (2001) 179–204.
- [36] A. Immonen, E. Niemelä, Survey of reliability and availability prediction methods from the architectural perspective, Software and Systems Modeling 7 (1) (2008) 49–65.
- [37] S. Becker, H. Koziol, R. Reussner, The Palladio component model for model-driven performance prediction, Journal of Systems and Software 82 (1) (2009) 3–22.
- [38] G. Franks, T. Al-Omari, M. Woodside, O. Das, S. Derisavi, Enhanced modeling and solution of layered queueing networks, IEEE Transactions on Software Engineering 35 (2) (2009) 148–161.
- [39] J. Delange, P. H. Feiler, Architecture fault modeling with the AADL error-model annex, in: 2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA), IEEE, 2014, pp. 361–368. doi:10.1109/SEAA.2014.20.
- [40] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, C. Delimitrou, Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices, in: Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2019.
- [41] Y. Gan, M. Liang, S. Dev, D. Lo, C. Delimitrou, Sage: Practical and scalable ML-driven performance debugging in microservices, in: Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2021.
- [42] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root cause localization of performance issues in microservices, in: Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS), 2020.
- [43] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, D. Pei, Practical root cause localization for microservice systems via trace analysis, in: Proc. IEEE/ACM Int. Symposium on Quality of Service (IWQoS), 2021.
- [44] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning, in: Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2022.
- [45] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, M. R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2023.
- [46] X. Meng, P. Shen, Y. Sun, D. Liu, J. Lu, S. Zhang, D. Pei, Microcause: Root cause analysis for microservice systems through graph neural networks, in: Proc. IEEE International Conference on Software Maintenance and Evolution (ICSME), 2020, pp. 403–414.
- [47] S. Zhang, S. Xia, W. Fan, B. Shi, X. Xiong, Z. Zhong, M. Ma, Y. Sun, D. Pei, Failure diagnosis in microservice systems: A comprehensive survey and analysis, arXiv preprint arXiv:2407.01710 (2024).
- [48] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, IEEE Transactions on Software Engineering 47 (2) (2021) 243–260.
- [49] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, IEEE Transactions on Software Engineering 22 (10) (1996) 751–761.
- [50] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: Proc. 27th Int. Conf. on Software Engineering (ICSE), 2005, pp. 580–586.
- [51] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE), 2007.
- [52] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, IEEE Transactions on Software Engineering 33 (1) (2007) 2–13.
- [53] V. Bushong, D. Das, A. Al Maruf, T. Cerny, Using static analysis to address microservice architecture reconstruction, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2021. doi:10.1109/ASE51524.2021.9678749.

- [54] S. Schneider, A. Bakhtin, X. Li, J. Soldani, A. Brogi, T. Cerny, R. Scandariato, D. Taibi, Comparison of static analysis architecture recovery tools for microservice applications, arXiv preprint (2024). [arXiv:2412.08352](#), [doi:10.48550/arXiv.2412.08352](#).
- [55] A. Santos, A. Cunha, N. Macedo, Statistical and model-driven static analysis of ROS systems, *IEEE Transactions on Software Engineering* 47 (10) (2019) 2200–2218.
- [56] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: *Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA)*, LNCS 5581, 2009, pp. 146–162.
- [57] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, *IEEE Software* 35 (3) (2018) 56–62.
- [58] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, *Journal of Systems and Software* 101 (2015) 193–220.
- [59] J. Humble, D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
- [60] L. Chen, Continuous delivery: Huge benefits, but challenges too, *IEEE Software* 32 (2) (2015) 50–54.
- [61] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).
- [62] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality measure elements, Tech. rep., International Organization for Standardization (2012).
- [63] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*, McGraw-Hill, 1980.
- [64] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, *Nature Machine Intelligence* 2 (2020) 317–324.
- [65] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: *Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2019, pp. 559–568.
- [66] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, Datasets and Benchmarks Track, 2024, arXiv:2402.02827.
- [67] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
- [68] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 1024–1034.
- [69] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2018.
- [70] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: *Proc. European Semantic Web Conference (ESWC)*, 2018, pp. 593–607.
- [71] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: *Proc. The Web Conference (WWW)*, 2019, pp. 2022–2032.
- [72] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2704–2710.
- [73] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2331–2341.
- [74] G. Khodabandeh, A. Ezaz, M. Babaei, N. Ezzati-Jivan, Utilizing graph neural networks for effective link prediction in microservice architectures, in: *Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE)*, 2025.
- [75] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32, 2019, pp. 9244–9255.
- [76] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, X. Zhang, Parameterized explainer for graph neural network, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, 2020, pp. 19620–19631.
- [77] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*, Morgan Kaufmann, 1988.
- [78] G. Beliakov, A. Pradera, T. Calvo, *Aggregation functions: A guide for practitioners*, Studies in Fuzziness and Soft Computing 221 (2007).
- [79] R. R. Yager, On ordered weighted averaging aggregation operators in multicriteria decisionmaking, *IEEE Transactions on Systems, Man, and Cybernetics* 18 (1) (1988) 183–190.
- [80] G. H. Hardy, J. E. Littlewood, G. Pólya, *Inequalities*, 2nd Edition, Cambridge University Press, 1952.
- [81] U.S. Department of Defense, MIL-STD-498: Software development and documentation, Military standard, U.S. Department of Defense (1994).
- [82] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [83] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: *Proc. 25th Int. Conf. on Machine Learning (ICML)*, 2008, pp. 1192–1199.
- [84] Team SimPy, Simpy: Discrete event simulation for Python, Software, <https://simpy.readthedocs.io> (accessed 9 September 2026) (2020).
- [85] F. Wilcoxon, Individual comparisons by ranking methods, *Biometrics Bulletin* 1 (6) (1945) 80–83.
- [86] B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall, 1993.
- [87] C. Spearman, The proof and measurement of association between two things, *American Journal of Psychology* 15 (1) (1904) 72–101.
- [88] R. Schwartz, J. Dodge, N. A. Smith, O. Etzioni, Green AI, *Communications of the ACM* 63 (12) (2020) 54–63. [doi:10.1145/3381831](#).
- [89] E. Strubell, A. Ganesh, A. McCallum, Energy and policy considerations for deep learning in NLP, in: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, Florence, Italy, 2019, pp. 3645–3650. [doi:10.18653/v1/P19-1355](#).
- [90] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, J. Dean, Carbon emissions and large neural network training, arXiv preprint arXiv:2104.10350 (2021). [doi:10.48550/arXiv.2104.10350](#).
- [91] S. Georgiou, M. Kechagia, T. Sharma, F. Sarro, Y. Zou, Green AI: Do deep learning frameworks have different costs?, in: *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, 2022, pp. 1082–1094. [doi:](#)

- 10.1145/3510003.3510221.
- [92] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, V. Prasanna, GraphSAINT: Graph sampling based inductive engine, in: Proc. International Conference on Learning Representations (ICLR), 2020.
- [93] I. O. Yigit, F. Buzluca, [dataset] software-as-a-graph: Replication package (datasets, generator configurations, simulation harnesses, model checkpoints, and analysis scripts), <https://doi.org/10.5281/zenodo.14922108> (2026). doi:10.5281/zenodo.14922108.